YATRASETU AI: AI-POWERED MULTILINGUAL SAFE-TRAVEL AND CROWD ADVISORY ASSISTANT FOR UTTARAKHAND

Minor Project Report

Submitted by

Ayush Singh (230240170024)
B.Tech CSE (AI/ML), 4th Year

In Partial Fulfillment of the Requirements
for the Degree of

Bachelor of Technology
in Computer Science and Engineering (AI/ML)



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
ROORKEE INSTITUTE OF TECHNOLOGY, ROORKEE
(Affiliated to VMSB Uttarakhand Technical University, Dehradun)

October 2026

DECLARATION

I declare that the work embodied in this Minor Project report entitled **"YatraSetu AI: AI-Powered Multilingual Safe-Travel and Crowd Advisory Assistant for Uttarakhand"** is my own original work carried out by me in the Department of Computer Science and Engineering for the session 2026-27 at **"Roorkee Institute of Technology, Roorkee"**. The matter embodied in this project report has not been submitted elsewhere for the award of any other degree. I declare that I have faithfully acknowledged, given credit to and referred to the researchers wherever the work has been cited in the text and the body of the report. I further certify that I have not wilfully lifted up some other's work, para, text, data, results, etc. reported in the journals, books, magazines, reports, dissertations, theses, etc., or available at websites and have included them in this project report and cited as my own work.

Date:

Place: Roorkee

Ayush Singh
Roll No. 230240170024
(Signature of the student)

Name & Signature of
Internal Examiner

Name & Signature of
External Examiner

ACKNOWLEDGEMENT

I am very happy to acknowledge the many people who helped make my project **"YatraSetu AI: AI-Powered Multilingual Safe-Travel and Crowd Advisory Assistant for Uttarakhand"** a successful one.

I take this opportunity to express my deep sense of gratitude to our honourable Director, **Dr Parag Jain**, for providing an excellent academic climate in the college that made this endeavour possible.

I give my wholehearted admiration and a deep sense of gratitude to **Dr Parul Choudhary**, HOD, **Department of Computer Science & Engineering**, Roorkee Institute of Technology, Roorkee, for the inspiration, valuable guidance, encouragement, suggestions and overall help throughout the project.

Finally, I express my gratitude to all the teaching and non-teaching staff of the **Department of Computer Science & Engineering**, Roorkee Institute of Technology, Roorkee, for their timely support and suggestions, and to my family and friends for their constant encouragement.

Date:

Ayush Singh
Roll No. 230240170024

ABSTRACT

Uttarakhand received a record 6.03 crore visitors in 2025, including more than 51 lakh Char Dham pilgrims. Pilgrims make safety-critical decisions about when to travel, which route to take and whether they are fit to trek, but the information they need is scattered across portals, apps, press notes and social media, is rarely personalised and is often not available in their language. In 2025, Kedarnath recorded a single-day peak of 30,154 pilgrims against a recommended carrying capacity of about 13,111, landslides repeatedly halted the yatra, and 56 pilgrims died in its first 28 days, mostly from cardiac causes.

This project presents **YatraSetu AI**, a low-cost, multilingual, AI-driven safe-travel and crowd advisory assistant. A bilingual (English and Hindi) assistant answers questions using retrieval-augmented generation over verified content, cites its sources and handles emergencies through a deterministic safety path. A gradient-boosted regression model forecasts daily footfall at the five shrines and compares it with carrying capacity. An explainable route-risk model combines weather alerts, rainfall, official road status and terrain susceptibility, and alerts subscribed travellers by WhatsApp with SMS fallback, suggesting a safe halt. A rule-based module checks health readiness, and an officials' dashboard supports bilingual advisories with two-person approval, analytics and an audit log.

The prototype is built with Python, FastAPI, scikit-learn and a Progressive Web App front-end, and runs fully offline, with optional Bhashini and LLM integration. Evaluation shows 95.0% top-3 retrieval accuracy on a 40-question bilingual gold set, 100% recall of emergency messages, 13.3% mean absolute percentage error for 7-day crowd forecasts on a held-out season, and all 18 automated tests passing.

Keywords: *Smart tourism, Retrieval-Augmented Generation, crowd forecasting, gradient boosting, landslide risk, multilingual AI, Uttarakhand, Char Dham.*

TABLE OF CONTENTS

Contents **Page No.**

DECLARATION ..... **i**

ACKNOWLEDGEMENT ..... **ii**

ABSTRACT ..... **iii**

LIST OF FIGURES ..... **vi**

LIST OF TABLES ..... **vii**

1. INTRODUCTION **1**

1.1 Background of the Project 1

1.2 Problem Statement 2

1.3 Project Objective 3

1.4 Advantages of this Project 3

1.5 Scope of Project 4

1.6 Tools & Technology Used 4

1.7 Organisation of the Report 5

2. ANALYSIS **6**

2.1 Study of the Existing System 6

2.2 Literature Review 6

2.3 Data Analysis 8

2.4 Requirement Analysis 10

2.5 Feasibility Analysis 10

2.6 Gaps in Reported and Existing Work, and Motivation 11

3. SOFTWARE REQUIREMENTS SPECIFICATION **13**

3.1 System Configurations 13

3.2 Introduction 14

3.3 Overall Description 15

3.4 Functional Requirements 16

3.5 Non-Functional Requirements 18

3.6 Use Case Model 19

3.7 Data Flow Diagrams 20

3.8 Entity Relationship Model and Data Dictionary 22

3.9 External Interface Requirements 23

4. TECHNOLOGY USED AND ITS DESCRIPTION **25**

4.1 System Architecture 25

4.2 Python and FastAPI 26

4.3 Machine Learning with scikit-learn 27

4.4 Retrieval-Augmented Generation, LLM Integration and Bhashini 29

4.5 Route-Risk Scoring Algorithm 30

4.6 Health-Readiness Rules 31

4.7 Database 31

4.8 Front-End: HTML5, CSS3, JavaScript and Progressive Web App....31
4.9 Messaging and Advisory Workflow....31
4.10 Security and Privacy....32
4.11 Testing Tools....32
5. CODING....33
5.1 Project Structure....33
5.2 Hybrid TF-IDF retriever (app/knowledge_base.py)....34
5.3 Intent classifier with emergency override (app/intent.py)....35
5.4 Conversation orchestrator (app/assistant.py)....35
5.5 Crowd forecasting model and evaluation (app/forecast.py)....37
5.6 Route-risk scoring and safe-halt search (app/risk.py)....38
5.7 Four-eyes approval and alert dispatch (app/alerts.py)....39
5.8 LLM grounding guardrail (app/llm.py)....39
5.9 Health-readiness rules (app/health.py)....40
5.10 REST API endpoints and role-based access (app/main.py)....41
5.11 Chat client in the traveller web app (app/static/app.js)....42
5.12 Integration test of the advisory workflow (tests/test_core.py)....42
6. SCREENSHOTS....44
6.1 Traveller Web App....44
6.2 Officials' Dashboard....49
6.3 Testing and Evaluation Results....53
7. CONCLUSION....56
7.1 Conclusion....56
7.2 Achievement of Objectives....56
7.3 Limitations....57
7.4 Future Scope....57
8. BIBLIOGRAPHY....58

LIST OF FIGURES

Figure 1.1: Total visitors to Uttarakhand, 2021 to 2025 (Tourism Department data reported by ETV Bharat, 2026a)..... 1
Figure 2.1: Daily limits per shrine compared with the observed Kedarnath peak (The Quint, 2022; ETV Bharat, 2026b; Counterview, 2025).....9
Figure 2.2: Project schedule (Gantt chart).....11
Figure 3.1: Use case diagram of YatraSetu AI.....19
Figure 3.2: Data flow diagram, Level 0 (context diagram).....20
Figure 3.3: Data flow diagram, Level 1.....21
Figure 3.4: Entity relationship diagram.....22
Figure 4.1: Layered architecture of YatraSetu AI.....25
Figure 4.2: Processing flow of a user message in the assistant.....29
Figure 4.3: Sequence diagram of advisory drafting, approval and broadcast.....32
Figure 6.1: Assistant answering registration, Hindi crowd and route questions.....44
Figure 6.2: Emergency override and off-topic fallback.....45
Figure 6.3: Crowd forecast for Kedarnath for the opening week of 2027 against carrying capacity.....45
Figure 6.4: Live route-risk map and explained segment scores for the Kedarnath route..46
Figure 6.5: Health-readiness self-check for a 62-year-old pilgrim with high BP and heart disease.....46
Figure 6.6: Trip subscription with consent, and SOS with nearest help.....47
Figure 6.7: Mobile view: Hindi question about helicopter booking.....48
Figure 6.8: Dashboard login (editor and approver roles).....49
Figure 6.9: Dashboard overview: KPIs, questions by intent and live route risk.....49
Figure 6.10: Editor drafts a bilingual advisory, which waits for approval.....50
Figure 6.11: Approver publishes the advisory: 4 recipients, 3 by WhatsApp and 1 by SMS fallback.....50
Figure 6.12: Road-status update and IMD district alert entry.....51
Figure 6.13: Alert delivery log in the traveller's language, with SMS fallback.....51
Figure 6.14: Forecast model back-test on the held-out 2025 season.....52
Figure 6.15: Audit log of official actions.....52
Figure 6.16: Automated test run: 18 tests passed.....54
Figure 6.17: Output of the offline evaluation script.....54
Figure 6.18: Evaluation results of the AI components.....55
Figure 6.19: Kedarnath forecast vs. actual on the held-out 2025 season (synthetic history) 55

LIST OF TABLES

Table 1.1: Project objectives and measurable targets....3
Table 1.2: Scope of the project....4
Table 1.3: Tools and technologies....4
Table 2.1: Existing systems and their limitations....6
Table 2.2: Summary of literature review....8
Table 2.3: Key figures used in the analysis....9
Table 2.4: Stakeholders and their needs....10
Table 2.5: Indicative deployment cost (INR)....10
Table 3.1: Definitions and abbreviations....14
Table 3.2: User classes....15
Table 3.3: Functional requirements....16
Table 3.4: Non-functional requirements....18
Table 3.5: Use case descriptions....19
Table 3.6: Data dictionary (main tables)....23
Table 3.7: Software interfaces....23
Table 4.1: Main REST API endpoints....26
Table 4.2: Forecast model features (all known 7 days in advance)....28
Table 4.3: Health-readiness rules....31
Table 5.1: Source modules....33
Table 6.1: Representative test cases....53
Table 6.2: Evaluation results....54
Table 7.1: Objectives versus achievement....56

CHAPTER 1
INTRODUCTION

1.1 Background of the Project

Uttarakhand, known as *Devbhoomi* (the land of the gods), is one of India's most visited Himalayan states. It is home to the Char Dham shrines (Yamunotri, Gangotri, Kedarnath and Badrinath), Hemkund Sahib, the pilgrim cities of Haridwar and Rishikesh, and hill stations such as Mussoorie and Nainital. According to Uttarakhand Tourism Department data reported by ETV Bharat (2026a), the state received **6,03,21,194 visitors in 2025**, the highest since its formation, including 1.93 lakh foreign tourists. Tourism has been estimated at roughly a quarter of the state's gross domestic product (Down To Earth), so it supports the livelihoods of drivers, hotel and homestay owners, porters, pony operators and small shopkeepers across the hills.

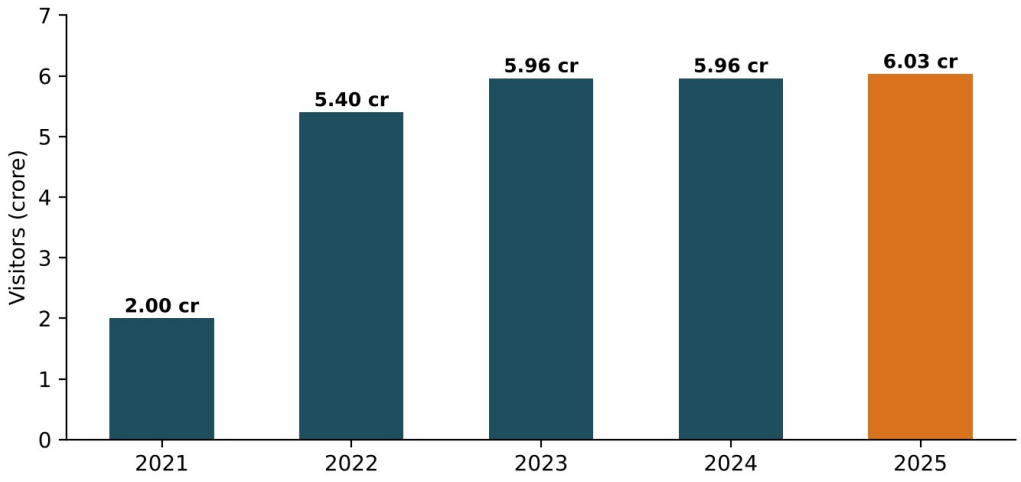


Figure 1.1: Total visitors to Uttarakhand, 2021 to 2025 (Tourism Department data reported by ETV Bharat, 2026a)

The Char Dham Yatra is the most concentrated and riskiest part of this flow. In 2025 it drew **51,06,346 pilgrims**, about 3.05 lakh more than in 2024 (Counterview, 2025), over a season of roughly six months, along narrow mountain highways and treks of up to about 18 km at altitudes above 3,000 m. The 2013 Kedarnath disaster showed how quickly extreme rainfall can turn a pilgrimage into a humanitarian crisis, and how much depends on timely information and coordination (Rautela, 2013). Rainfall-triggered landslides remain one of the deadliest hazards in the Himalaya (Froude and Petley, 2018).

At the same time, digital technology has changed what is possible. The idea of *smart tourism* uses information and communication technology to give travellers context-aware services and to help destinations manage visitor flows (Gretzel et al., 2015).

Transformer-based language models (Vaswani et al., 2017; Brown et al., 2020) can hold natural conversations, and Retrieval-Augmented Generation (RAG) lets such models answer from a curated knowledge source instead of from memory (Lewis et al., 2020). India's national language platform Bhashini and open models such as IndicTrans2 (Gala et al., 2023) make translation and speech services available for Indian languages. Machine-learning forecasting methods (Taylor and Letham, 2018; Ke et al., 2017) can predict demand from historical patterns.

YatraSetu AI (*setu* means bridge) brings these building blocks together in a single, low-cost system for Uttarakhand: a multilingual assistant for pilgrims, a crowd-forecasting service, a route-risk and alert service, a health-readiness module, and a dashboard for district officials. This report describes the analysis, requirements, design, implementation and evaluation of a working prototype of the system.

1.2 Problem Statement

Field reports, official data and expert analysis point to one underlying failure: **travellers make high-stakes decisions (when to go, which route to take, whether they are fit to trek) without timely, trustworthy and personalised information in a language they understand**, while authorities lack forecasting and targeted-broadcast tools to spread crowds and warn people early. It shows up as five pain points:

1. **Fragmented information (P1).** Registration, weather warnings, road status, helicopter booking, medical posts and shrine timings are spread across separate portals, apps, press notes and social media posts. No single channel answers a traveller's actual question.
2. **Unpredictable, over-capacity crowding (P2).** Kedarnath recorded **30,154 pilgrims on 2 May 2025**, while a 2025 carrying-capacity study recommended about **13,111 per day** (ETV Bharat, 2026b). The same season had 86 zero-pilgrim shrine-days caused by weather and closures (Counterview, 2025).
3. **Weather and landslide disruptions (P3).** On 3 July 2025 heavy rain triggered landslides and the yatra was halted (News On AIR, 2025). In September 2025 about 2,500 tourists were stranded in Mussoorie after roads and bridges were damaged (Business Today, 2025).
4. **Preventable health emergencies (P4).** 56 pilgrims died in the first 28 days of the 2025 yatra, mostly from cardiac causes and mostly people over 50 (ETV Bharat, 2025).
5. **Language, literacy and connectivity barriers (P5).** Pilgrims come from every state, many are elderly, and network coverage along routes is uneven, so English-first, text-heavy apps exclude the people at highest risk.

Problem statement

Design and build a low-cost, multilingual, AI-driven assistant that gives each Char Dham traveller verified, personalised guidance on when to travel, which route is safe and how to prepare, and that gives district officials crowd forecasts and a controlled channel to publish advisories and alert travellers on affected routes, using existing public data and open AI components.

1.3 Project Objective

The aim of the project is to design, implement and evaluate a working prototype of YatraSetu AI. The specific objectives are:

Table 1.1: Project objectives and measurable targets

ID	Objective	Measurable target
O1	Build a bilingual (English and Hindi) conversational assistant that answers common yatra questions from verified content with sources	Top-3 retrieval accuracy of 90% or more on a 40-question gold set; every answer shows its source
O2	Forecast daily footfall at the five shrines and compare it with carrying capacity	MAPE of 20% or less on normal days of a held-out season
O3	Score route-segment risk from weather alerts, road status and terrain, and alert subscribed travellers	Alerts generated automatically for Orange or Red segments, with SMS fallback
O4	Provide a rule-based health-readiness self-check and SOS	Correct category for all test profiles; nearest help found from GPS position
O5	Provide an officials' dashboard with two-person (four-eyes) approval of advisories	Author cannot approve own advisory; approved advisory immediately answerable by the assistant
O6	Keep the design low-cost, private and replicable	Runs offline on a laptop; consent and hashing of phone numbers; open-source stack

1.4 Advantages of this Project

- **Safety first:** emergencies are detected deterministically and answered with 112/108 instructions without depending on an AI model.
- **Trustworthy answers:** every answer is grounded in approved official content or live data, and the source and last-updated date are shown.
- **Inclusive:** works in Hindi and English out of the box, with an adapter for other Indian languages through Bhashini; designed for WhatsApp and SMS, which most pilgrims already use.

- **Better crowd distribution:** a 7-day outlook against carrying capacity helps pilgrims pick lighter days and helps officials plan staff and parking.
- **Targeted warnings:** only travellers whose route actually crosses an affected segment are alerted, with a suggested safe halt.
- **Accountable governance:** four-eyes approval, role-based access and an audit log of every advisory and road-status change.
- **Low cost and easy to build:** no new hardware; mainstream open-source tools; the full prototype runs on a laptop.

1.5 Scope of Project

The scope of the prototype is defined below. Items marked out of scope are either already handled by official systems or need hardware, partnerships or data access beyond a student project.

Table 1.2: Scope of the project

In scope	Out of scope
Char Dham and Hemkund Sahib routes from Rishikesh (19 towns, 18 route segments)	Statewide tourism (hill stations, wildlife parks); planned as future work
Bilingual assistant (web app, WhatsApp webhook); Bhashini adapter for other languages	Voice calls (IVR) and Garhwali/Kumaoni speech recognition
Crowd forecasting with a trained model; carrying-capacity bands; alternative dates	CCTV, drone or sensor-based live crowd counting
Route-risk scoring; road-status and IMD-alert entry; safe-halt suggestion	Issuing official warnings (the system amplifies, never replaces, IMD/USDMA)
Trip subscription with consent; WhatsApp alerts with SMS fallback (simulated gateway)	Yatra registration, payments, helicopter or hotel booking
Health readiness self-check and SOS with nearest help	Medical diagnosis or treatment advice
Officials' dashboard: advisories, approvals, analytics, model metrics, audit log	Integration with live government databases (needs a data-sharing agreement)

1.6 Tools & Technology Used

Table 1.3: Tools and technologies

Category	Tool / technology	Use in the project
Programming language	Python 3.11+, JavaScript (ES2020)	Back-end, machine learning, front-end logic
Web framework	FastAPI, Uvicorn, Pydantic	REST API, request validation, automatic API documentation

Category	Tool / technology	Use in the project
Machine learning	scikit-learn, NumPy, pandas	TF-IDF retrieval, logistic-regression intent model, gradient-boosting forecaster
Language AI	Bhashini API (optional), any OpenAI-compatible LLM API (optional)	Translation for more Indian languages; generative RAG answers
Database	SQLite (prototype); PostgreSQL + PostGIS (production design)	Subscriptions, advisories, outbox, logs, audit
Front-end	HTML5, CSS3, JavaScript, Canvas, SVG, Service Worker	Progressive Web App, charts and route-risk map without external libraries
Messaging	WhatsApp Business Cloud API (webhook), SMS gateway	Inbound questions and outbound alerts (simulated outbox in prototype)
Testing	pytest, FastAPI TestClient, Playwright	Unit and API tests, automated screenshots
Design and documentation	Graphviz, Matplotlib, MS Word	UML/DFD/ER diagrams, charts, report
Version control and IDE	Git, Visual Studio Code	Source management and development

1.7 Organisation of the Report

Chapter 2 analyses the existing systems and literature, studies the available data and identifies the gaps that motivate the work. Chapter 3 is the Software Requirements Specification with system configuration, functional and non-functional requirements and UML/DFD/ER models. Chapter 4 describes the architecture, technologies and algorithms. Chapter 5 presents the code organisation and key listings. Chapter 6 shows screenshots of the working system together with test and evaluation results. Chapter 7 concludes with achievements, limitations and future scope, and Chapter 8 lists the bibliography.

CHAPTER 2
ANALYSIS

This chapter studies the systems that pilgrims and officials use today, reviews the relevant research, analyses the available data, examines feasibility and concludes with the gaps in existing work that motivate YatraSetu AI.

2.1 Study of the Existing System

Information for Char Dham travellers is currently provided by several independent systems, each covering one function:

Table 2.1: Existing systems and their limitations

Existing system	What it provides	Limitation for the traveller
Registration & Tourist Care portal and Tourist Care Uttarakhand app (Government of Uttarakhand, 2024)	Mandatory yatra registration, QR code, some tourist information	Transactional; does not answer free-form questions, forecast crowds or give route-specific warnings
Uttarakhand Bhookamp Alert app (USDMA) (Deccan Herald, 2021)	Earthquake early warning	Single hazard; not linked to the traveller's route or trip dates
IMD district warnings and USDMA bulletins	Colour-coded district rainfall alerts	District-level, technical; not translated into "what should I do on my route"
Police and district press notes, social media posts	Road closures, traffic diversions	Scattered, often only in Hindi or English, easy to miss; no targeting
Helplines (112, 108, tourist helplines)	Emergency and general help	Overloaded during peaks; no self-service
Private travel apps and websites	Bookings, generic guides	Not authoritative; fake helicopter-booking sites are a known fraud risk

The common weakness is that none of these systems combines **verified information + personalisation (route, date, age, language) + prediction + targeted alerts**. Officials also have no single place to publish an advisory once and have it reach every affected traveller in their language.

2.2 Literature Review

2.2.1 Smart tourism

Gretzel et al. (2015) define smart tourism as tourism supported by integrated efforts at a destination to collect and aggregate data from physical infrastructure, social connections and organisational sources, and to use advanced technologies to turn that data into on-site experiences and business value. They stress real-time, context-aware

services for travellers and data-driven management for destinations. YatraSetu applies this idea to pilgrimage safety rather than only to marketing or convenience.

2.2.2 Language models and Retrieval-Augmented Generation

The Transformer architecture (Vaswani et al., 2017) is the basis of modern language models, and Brown et al. (2020) showed that very large models can perform many tasks from a few examples. Such models can, however, produce fluent but unsupported statements; Ji et al. (2023) survey this *hallucination* problem and its mitigation. Lewis et al. (2020) proposed Retrieval-Augmented Generation, which retrieves relevant passages from an external corpus and conditions generation on them, improving factuality and allowing the knowledge to be updated without retraining. Classical retrieval models such as TF-IDF weighting (Salton and Buckley, 1988) and BM25 (Robertson and Zaragoza, 2009) remain strong, transparent baselines (Manning et al., 2008). For safety-critical advice, YatraSetu therefore uses RAG with strict grounding and a deterministic fallback.

2.2.3 Indian language technology

Gala et al. (2023) released IndicTrans2, open machine-translation models covering all 22 scheduled Indian languages. The Government of India's Bhashini platform exposes translation for 22 languages and speech recognition and synthesis for 12 languages through APIs (UNICEF). These resources make a multilingual assistant feasible without training new models.

2.2.4 Demand forecasting

Hyndman and Athanasopoulos (2021) describe forecasting methods from exponential smoothing to regression with calendar effects. Taylor and Letham (2018) introduced Prophet, an additive model with trend, seasonality and holiday components, for forecasting at scale. Gradient boosting (Friedman, 2001) and its efficient histogram-based implementations such as LightGBM (Ke et al., 2017) are widely used for tabular prediction because they capture non-linear interactions between features such as season position, weekday and weather. YatraSetu uses histogram gradient boosting on features known in advance.

2.2.5 Landslide hazard and early warning

Froude and Petley (2018) compiled 4,862 fatal non-seismic landslides between 2004 and 2016 and identified the Himalayan arc as a global hotspot, with most events triggered by rainfall. Kirschbaum and Stanley (2018) built a satellite-rainfall-based model (LHASA) that combines rainfall with a static susceptibility map to give near-real-time landslide hazard for situational awareness. Rautela (2013) analysed the 2013

Kedarnath disaster and highlighted gaps in warning, communication and regulation of pilgrim flows. YatraSetu's route-risk score follows the same principle of combining dynamic rainfall/alert information with static susceptibility, at the scale of road segments.

2.2.6 Software requirements engineering

The requirements in this project follow IEEE Std 830-1998 and its successor ISO/IEC/IEEE 29148:2018, and the process models described by Pressman and Maxim (2020) and Sommerville (2016).

Table 2.2: Summary of literature review

Author (Year)	Focus	Key contribution	Use / gap for this project
Gretzel et al. (2015)	Smart tourism	Framework for data-driven, real-time tourist services	Applied to pilgrimage safety; no Himalayan pilgrimage system found
Vaswani et al. (2017); Brown et al. (2020)	Language models	Transformer; few-shot learning	Enable natural-language assistant
Lewis et al. (2020)	RAG	Retrieval + generation for factual answers	Core answer method; needs domain knowledge base
Ji et al. (2023)	Hallucination	Survey of causes and mitigation	Motivates grounding guardrail and fallback
Gala et al. (2023); Bhashini	Indian languages	Translation for 22 languages	Multilingual access for pilgrims
Taylor and Letham (2018); Ke et al. (2017)	Forecasting	Seasonal models; gradient boosting	Shrine footfall forecasting
Froude and Petley (2018); Kirschbaum and Stanley (2018)	Landslides	Global inventory; rainfall + susceptibility hazard model	Route-segment risk score
Rautela (2013)	Kedarnath 2013	Lessons on warning and pilgrim management	Need for early, targeted communication

2.3 Data Analysis

Publicly reported data was analysed to size the problem and to set design parameters.

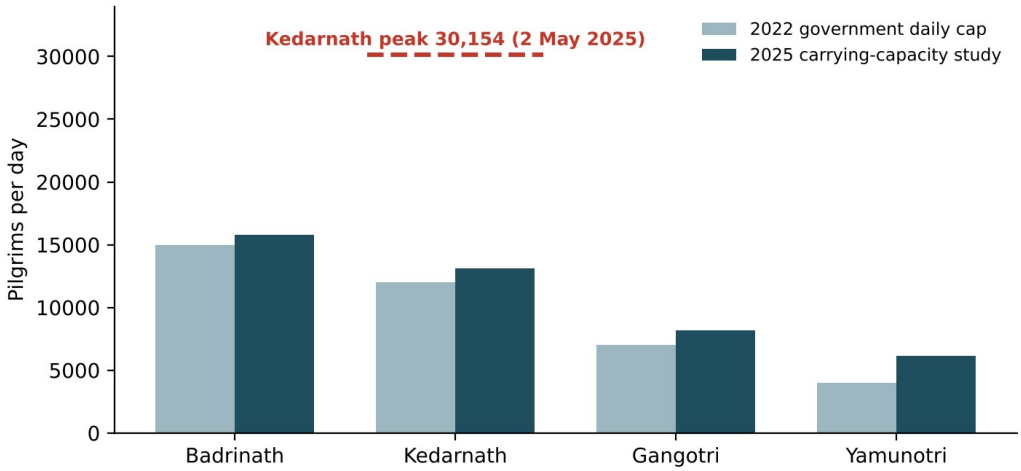


Figure 2.1: Daily limits per shrine compared with the observed Kedarnath peak (The Quint, 2022; ETV Bharat, 2026b; Counterview, 2025)

Table 2.3: Key figures used in the analysis

Indicator	Value	Source
Visitors to Uttarakhand, 2025	6,03,21,194	ETV Bharat (2026a)
Char Dham pilgrims, 2025 (2024)	51,06,346 (48,01,167)	Counterview (2025)
Highest single-day footfall, 2025	30,154 at Kedarnath, 2 May 2025	Counterview (2025)
Zero-pilgrim shrine-days, 2025	86	Counterview (2025)
Recommended daily capacity (2025 study)	Badrinath 15,778; Kedarnath 13,111; Gangotri 8,178; Yamunotri 6,160	ETV Bharat (2026b)
Pilgrim deaths, first 28 days of 2025 yatra	56 (Kedarnath route 26), mostly cardiac	ETV Bharat (2025)
Registrations by 21 June 2025	About 45 lakh	The Week (2025)
Tourists stranded in Mussoorie, Sept 2025	About 2,500	Business Today (2025)

Three conclusions follow. First, demand is highly peaked: the opening weeks and weekends exceed carrying capacity by a large margin, so even modest shifting of demand to lighter days has value. Second, disruption is frequent and route-specific, so warnings must be tied to the traveller's segment and dates. Third, the people most at risk are elderly pilgrims from many language backgrounds, so the interface must be simple, multilingual and work on WhatsApp and SMS.

2.4 Requirement Analysis

Table 2.4: Stakeholders and their needs

Stakeholder	Need	How YatraSetu responds
Pilgrim / tourist	Reliable answers, best day to travel, route safety	Assistant, crowd forecast, route risk, alerts
Elderly / low-literacy traveller	Simple, own-language, voice-friendly help	Hindi/English, quick-reply buttons, Bhashini adapter
District official (editor)	Publish road and weather advisories quickly	Advisory drafting with bilingual text, road-status entry
Senior official (approver)	Control over what is broadcast	Four-eyes approval, audit log
Tourism department	Insight into traveller concerns and crowding	Query analytics, forecast metrics
Health department	Screening compliance for 50+ pilgrims	Screening reminders and medical-post finder

2.5 Feasibility Analysis

2.5.1 Technical feasibility

All components are mature and available as open-source libraries or public APIs: FastAPI and scikit-learn for the back-end and models, Bhashini for Indian languages, and standard web technologies for the front-end. No new hardware or model training at scale is required. The prototype in this report demonstrates technical feasibility directly.

2.5.2 Economic feasibility

The prototype was built with free, open-source software on a personal laptop. For a state-level deployment, indicative costs are shown below (estimates at typical market rates, not quotations).

Table 2.5: Indicative deployment cost (INR)

Item	Basis	Amount
Development team (6 months)	ML engineer, full-stack and front-end developers, part-time UX/QA and lead	about 19.1 lakh
Cloud, field testing, security audit, contingency	Development environment, user trials on route, VAPT, 10% contingency	about 5.7 lakh
Annual operations	Hosting, LLM API, WhatsApp/SMS messages, one maintenance engineer	about 21 lakh per year
Cost per pilgrim per season	21 lakh / 51 lakh pilgrims	about 0.41

2.5.3 Operational feasibility

Pilgrims already use WhatsApp and must already register digitally, so no new app installation is needed. The main operational dependency is that district control rooms enter road status and approve advisories; the dashboard workflow is designed to take only a few minutes. A data-sharing agreement is needed to replace the synthetic footfall history with official counts.

2.5.4 Schedule feasibility

The project was planned over a 16-week semester as shown below and completed within the schedule.

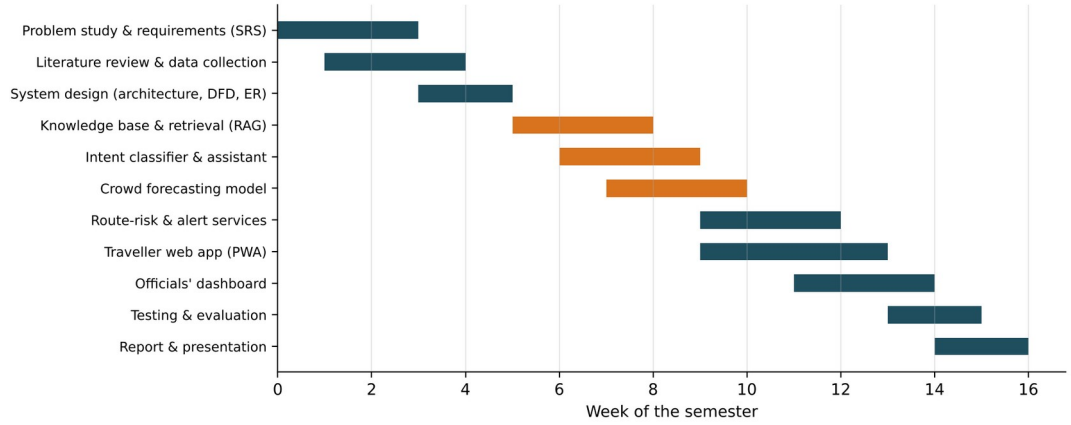


Figure 2.2: Project schedule (Gantt chart)

2.6 Gaps in Reported and Existing Work, and Motivation

From the study of existing systems, the literature and the data, the following gaps were identified:

1. **No single, conversational source of verified yatra information.** Existing apps are transactional or single-hazard; travellers must search across many sources.
2. **No public crowd forecast linked to carrying capacity.** Capacity studies and daily counts exist, but travellers cannot see in advance which days will be over capacity.
3. **Warnings are district-level, not route- and date-specific.** Research on landslide hazard combines rainfall with susceptibility, but this is not turned into personalised advice for travellers on a road segment.
4. **Language and accessibility gaps.** Most services are English or Hindi only and text-heavy, although Indian-language AI is now available through Bhashini and open models.

5. **Unsafe use of generative AI.** General-purpose chatbots can hallucinate; safety-critical travel advice needs grounding, citations and a deterministic emergency path.

Motivation. As a student of an Uttarakhand institution specialising in AI/ML, the author was motivated by the repeated news of stranded and ill pilgrims during each yatra season. The project shows that a small, well-designed system built from open components can close these gaps: it gives every pilgrim a trusted assistant in their language, helps spread crowds, and gives officials a fast, accountable way to warn exactly the people who need it.

CHAPTER 3
SOFTWARE REQUIREMENTS SPECIFICATION

This chapter specifies YatraSetu AI following the structure of IEEE Std 830-1998 and ISO/IEC/IEEE 29148:2018. It refines the objectives of Chapter 1 into a complete information description, functional and behavioural requirements, performance and design constraints, and validation criteria.

3.1 System Configurations

The software requirement specification is produced at the end of the analysis task. The function and performance allocated to the software are refined by establishing a complete information description, a detailed functional description, a representation of system behaviour, performance and design constraints, and validation criteria.

3.1.1 Software requirements

- **Operating system:** Windows 10/11, macOS 12 or later, or Ubuntu 22.04 LTS
- **Coding language:** Python 3.11 or later (back-end, ML); JavaScript ES2020 (front-end)
- **Framework:** FastAPI 0.110+ with Uvicorn ASGI server and Pydantic validation
- **Machine learning libraries:** scikit-learn 1.4+, pandas 2.x, NumPy
- **Front-end:** HTML5, CSS3, JavaScript (Progressive Web App, Canvas and SVG charts)
- **Database:** SQLite 3 (prototype); PostgreSQL 15 with PostGIS (production design)
- **Browser:** Google Chrome 90+, Safari 15+, Firefox 90+ (desktop or mobile)
- **Development tools:** Visual Studio Code, Git, pytest, Playwright
- **Optional services:** Bhashini API credentials; an OpenAI-compatible LLM API key; WhatsApp Business Platform account

3.1.2 Hardware requirements

- **Development system:** Intel Core i5 (8th gen) / AMD Ryzen 5 / Apple M1 or better
- **RAM:** 8 GB (4 GB minimum)
- **Hard disk:** 1 GB free space for code, virtual environment and data
- **Production server (recommended):** 4 vCPU, 8 GB RAM, 50 GB SSD, auto-scaling during the yatra season

- **Client device:** any Android 8+ or iOS 14+ smartphone with WhatsApp or a browser; any mobile phone for SMS alerts

3.2 Introduction

3.2.1 Purpose

This SRS describes the functional, interface, data and quality requirements of YatraSetu AI version 1.0. It is intended for developers, testers, evaluators and the potential sponsor (Uttarakhand Tourism Development Board and district administrations).

3.2.2 Product scope

YatraSetu AI is a multilingual assistant and advisory platform for Char Dham and Hemkund Sahib travellers. It provides grounded question answering, crowd forecasts, route-risk alerts, health-readiness guidance and SOS, together with a dashboard for officials to publish verified advisories. Registration, booking and medical diagnosis are out of scope.

3.2.3 Definitions, acronyms and abbreviations

Table 3.1: Definitions and abbreviations

Term	Meaning
RAG	Retrieval-Augmented Generation: answers are produced only from retrieved, approved passages
LLM	Large Language Model
TF-IDF	Term Frequency-Inverse Document Frequency text representation
Bhashini	Government of India national language-technology platform (translation, ASR, TTS)
PWA	Progressive Web App: installable web app with offline caching
Carrying capacity	Recommended maximum daily pilgrims a shrine can safely host
Advisory	Official notice (road, weather, crowd, health) approved by an authorised official
Route segment	Stretch of road or trek between two named towns, e.g. Guptkashi to Sonprayag
Four-eyes rule	An advisory must be approved by a different person from its author
MAPE	Mean Absolute Percentage Error
IMD / USDMA	India Meteorological Department / Uttarakhand State Disaster Management Authority
RBAC	Role-Based Access Control

3.3 Overall Description

3.3.1 Product perspective

YatraSetu AI is a new, self-contained system that works **alongside** existing government systems. It reads aggregated footfall data, weather alerts and road status, and it links travellers to official portals for registration and booking. It does not write to any government system.

3.3.2 Product functions

- Answer travel, registration, weather, route, shrine and health-preparedness questions in the user's language, with sources.
- Show a 7-day crowd outlook for each shrine and suggest lighter dates or an alternative shrine.
- Score the risk of every route segment and show it on a map with explanations; suggest a safe halt.
- Let users subscribe a trip with consent and receive alerts by WhatsApp with SMS fallback.
- Run a health-readiness self-check and show screening points and medical posts on the route.
- Provide an SOS function that shares location and finds the nearest help.
- Let officials draft, approve and broadcast advisories, update road status and weather alerts, and view analytics, delivery logs, model metrics and the audit log.

3.3.3 User classes and characteristics

Table 3.2: User classes

User class	Characteristics	Main functions used
Pilgrim / tourist	Any age, any Indian language, varied digital literacy, often on low bandwidth	Assistant, crowd forecast, route risk, health check, alerts, SOS
Editor (district official)	Control-room staff; drafts advisories and updates road status	Advisories, road status and weather, overview
Approver (senior official, e.g. DDMO)	Authorises what is broadcast	Approval, delivery log, audit log
Administrator / analyst	Maintains knowledge base, reviews analytics and model quality	Analytics, forecast-model page

3.3.4 Operating environment

The server runs as a Python ASGI application on Linux, macOS or Windows, or in a container on a cloud platform such as MeghRaj (GI Cloud). Clients use any modern mobile or desktop browser, WhatsApp, or SMS.

3.3.5 Design and implementation constraints

- Answers to factual questions must come only from approved content or live service data; no free-form generation of safety facts.
- Emergency messages must be handled by deterministic rules, never by a probabilistic model.
- Every broadcast must have a named approver who is different from the author.
- Personal data must be minimised, stored in India and processed only with consent (DPDP Act, 2023).
- The front-end must work on low bandwidth and must not depend on external CDNs.

3.3.6 Assumptions and dependencies

Aggregated daily footfall counts can be obtained from the tourism department under a data-sharing agreement; until then a synthetic history calibrated to published season totals is used. Officials will enter road status during the season. Bhashini, WhatsApp Business Platform and LLM APIs remain available for the optional integrations.

3.4 Functional Requirements

Each requirement is given a priority using the MoSCoW scheme (M = Must, S = Should, C = Could) and its status in the prototype.

Table 3.3: Functional requirements

ID	Requirement	Pri.	Prototype status
FR-01	Accept text questions via web app and WhatsApp webhook; detect language automatically, with manual override	M	Implemented
FR-02	Accept voice notes and reply with speech (Bhashini ASR/TTS)	S	Planned (adapter ready)
FR-03	Answer from the approved knowledge base using retrieval and show source and last-updated date	M	Implemented
FR-04	Give a safe fallback with helpline/112 when retrieval confidence is low or the question is off-topic	M	Implemented
FR-05	Detect emergencies deterministically and return safety instructions and SOS	M	Implemented
FR-06	Provide quick-reply buttons for common questions	M	Implemented

ID	Requirement	Pri.	Prototype status
FR-07	Deep-link to official portals instead of performing registration or booking	M	Implemented (content)
FR-08	Load daily footfall history per shrine	M	Implemented (CSV)
FR-09	Forecast daily footfall 7 days ahead for five shrines	M	Implemented
FR-10	Classify each day as Low / Moderate / High / Over capacity against carrying capacity	M	Implemented
FR-11	Suggest up to two lighter dates and one alternative shrine for crowded days	S	Implemented
FR-12	Accept IMD/USDMA district alerts and rainfall (feed or manual entry)	M	Implemented
FR-13	Let officials mark a segment Open, Restricted or Closed with reason and expected reopening	M	Implemented
FR-14	Compute a 0-100 risk score and Green/Yellow/Orange/Red level per segment with explanations	M	Implemented
FR-15	Let users subscribe a trip with explicit consent	M	Implemented
FR-16	Notify affected subscribers by WhatsApp with SMS fallback	M	Implemented (simulated gateway)
FR-17	Include the nearest safe halt with facilities in every route alert	S	Implemented
FR-18	Run a health self-check and return a readiness category with guidance (no diagnosis)	M	Implemented
FR-19	Remind pilgrims aged 50+ about mandatory screening and list screening points on the route	M	Implemented
FR-20	List medical posts on the route and give acclimatisation tips	S	Implemented
FR-21	One-tap SOS sharing location and opening the 112 dialler; find nearest help	M	Implemented
FR-22	Cache the app shell for offline use	S	Implemented (service worker)
FR-23	Role-based login for officials (editor, approver)	M	Implemented
FR-24	Editors draft advisories with English and Hindi text, segment and validity	M	Implemented
FR-25	Only a different approver can publish; publishing updates the knowledge base and broadcasts alerts	M	Implemented
FR-26	Dashboard map of route risk and forecast vs. capacity	M	Implemented
FR-27	Analytics on questions by intent and language, grounded-answer rate	S	Implemented
FR-28	Show forecast-model accuracy and back-test chart	S	Implemented
FR-29	Record an audit log of logins, advisories, approvals, road and weather changes and SOS	M	Implemented
FR-30	Export daily and seasonal reports (PDF/CSV)	C	Planned

3.5 Non-Functional Requirements

Table 3.4: Non-functional requirements

ID	Attribute	Requirement	Prototype measurement
NFR-01	Performance	Text answer within 4 s (95th percentile)	About 5 ms retrieval + intent per query on a laptop
NFR-02	Alert latency	Alerts dispatched within 5 minutes of approval	Dispatched in the same request as approval
NFR-03	Availability	99.5% during the yatra season	Design target (stateless API, DB backups)
NFR-04	Scalability	5 lakh daily users at peak	Design target (horizontal scaling)
NFR-05	Graceful degradation	If LLM/Bhashini fail, extractive answers and alerts still work	Implemented; offline mode is the default
NFR-06	Localisation	Hindi and English; more languages via Bhashini	Hindi and English end to end
NFR-07	Accessibility	WCAG 2.1 AA (W3C, 2018); large text; simple flows	Responsive layout, 15-16 px text, high contrast
NFR-08	Low bandwidth	No external CDN; small payload	No third-party front-end libraries
NFR-09	Privacy	Consent, minimisation, deletion (DPDP Act, 2023)	Consent required; phone stored as salted SHA-256 hash + masked number
NFR-10	Security	Authentication, RBAC, input validation, audit	Token login, roles, Pydantic validation, audit log
NFR-11	Answer accuracy	Top-3 retrieval accuracy of 90% or more	95.0% on 40-question gold set
NFR-12	Hallucination control	No unsupported numbers in generated answers	Numeric grounding check + fallback
NFR-13	Forecast accuracy	MAPE of 20% or less on normal days	13.3% on held-out 2025 season
NFR-14	Maintainability	Modular code, automated tests	11 modules; 18 automated tests passing
NFR-15	Portability	Runs on any OS / cloud	Pure Python + browser; one-command setup script

3.6 Use Case Model

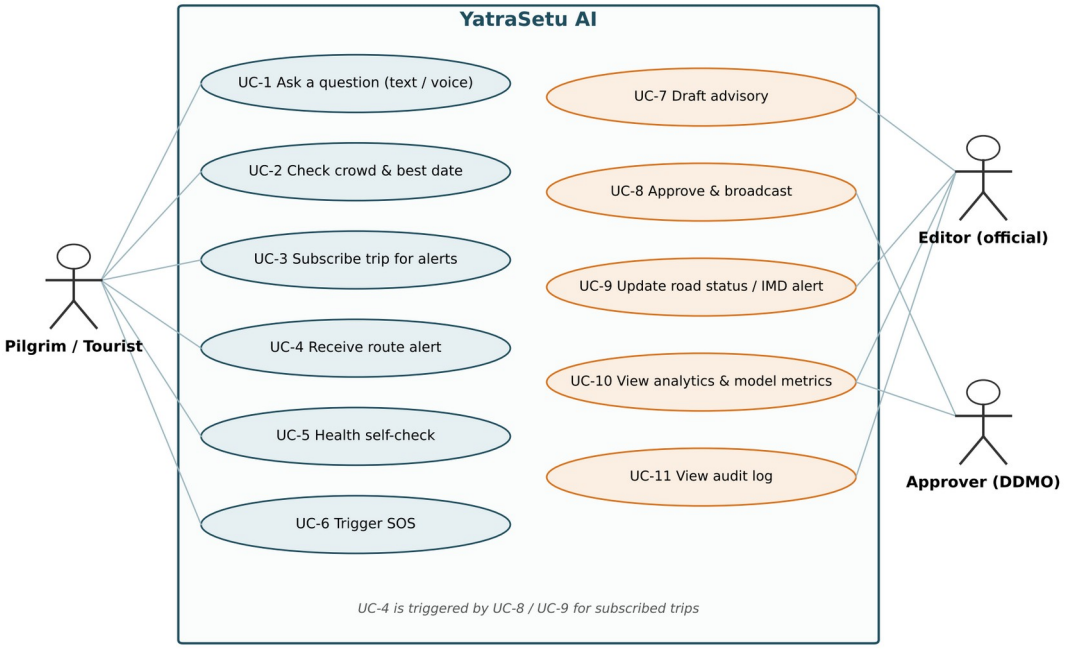


Figure 3.1: Use case diagram of YatraSetu AI

Table 3.5: Use case descriptions

ID	Use case	Actor	Main success scenario
UC-1	Ask a question	Pilgrim	Types a question in Hindi or English; receives a grounded answer with sources
UC-2	Check crowd	Pilgrim	Selects shrine and date; sees 7-day forecast, bands and alternatives
UC-3	Subscribe trip	Pilgrim	Enters phone, shrine, dates and gives consent; gets confirmation
UC-4	Receive alert	Pilgrim	Receives WhatsApp (or SMS) alert with reason, safe halt and 112
UC-5	Health self-check	Pilgrim	Enters age, conditions, symptoms; gets readiness category and route medical posts
UC-6	Trigger SOS	Pilgrim	Presses SOS; location message, 112 link and nearest help are shown
UC-7	Draft advisory	Editor	Enters title, English and Hindi text, segment, validity; advisory saved as pending
UC-8	Approve and broadcast	Approver	Approves pending advisory; knowledge base updated; affected subscribers alerted
UC-9	Update road status / IMD alert	Editor / Approver	Marks segment closed or enters district alert; risk scores and alerts update
UC-10	View analytics and model	Official	Views question statistics, delivery log, forecast accuracy
UC-11	View audit log	Official	Reviews every action with actor and time

3.7 Data Flow Diagrams

The context diagram (Level 0) shows YatraSetu AI as a single process exchanging data with pilgrims, officials and external data sources. Level 1 decomposes it into five processes and five data stores.

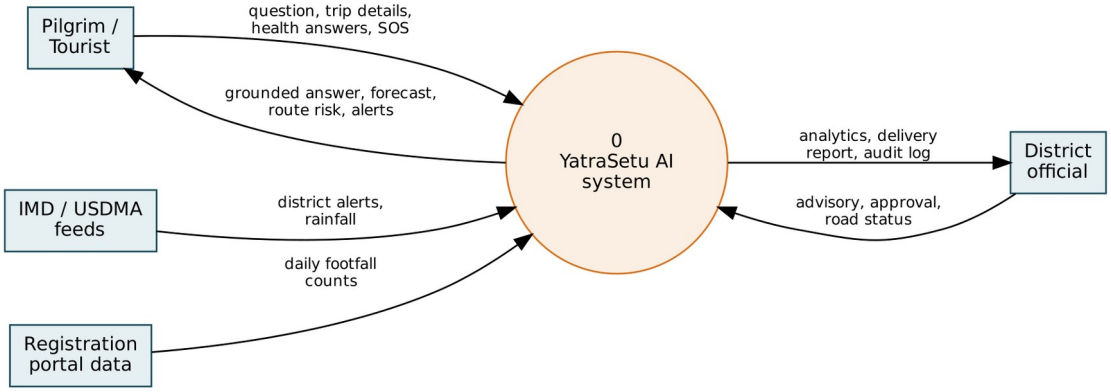


Figure 3.2: Data flow diagram, Level 0 (context diagram)

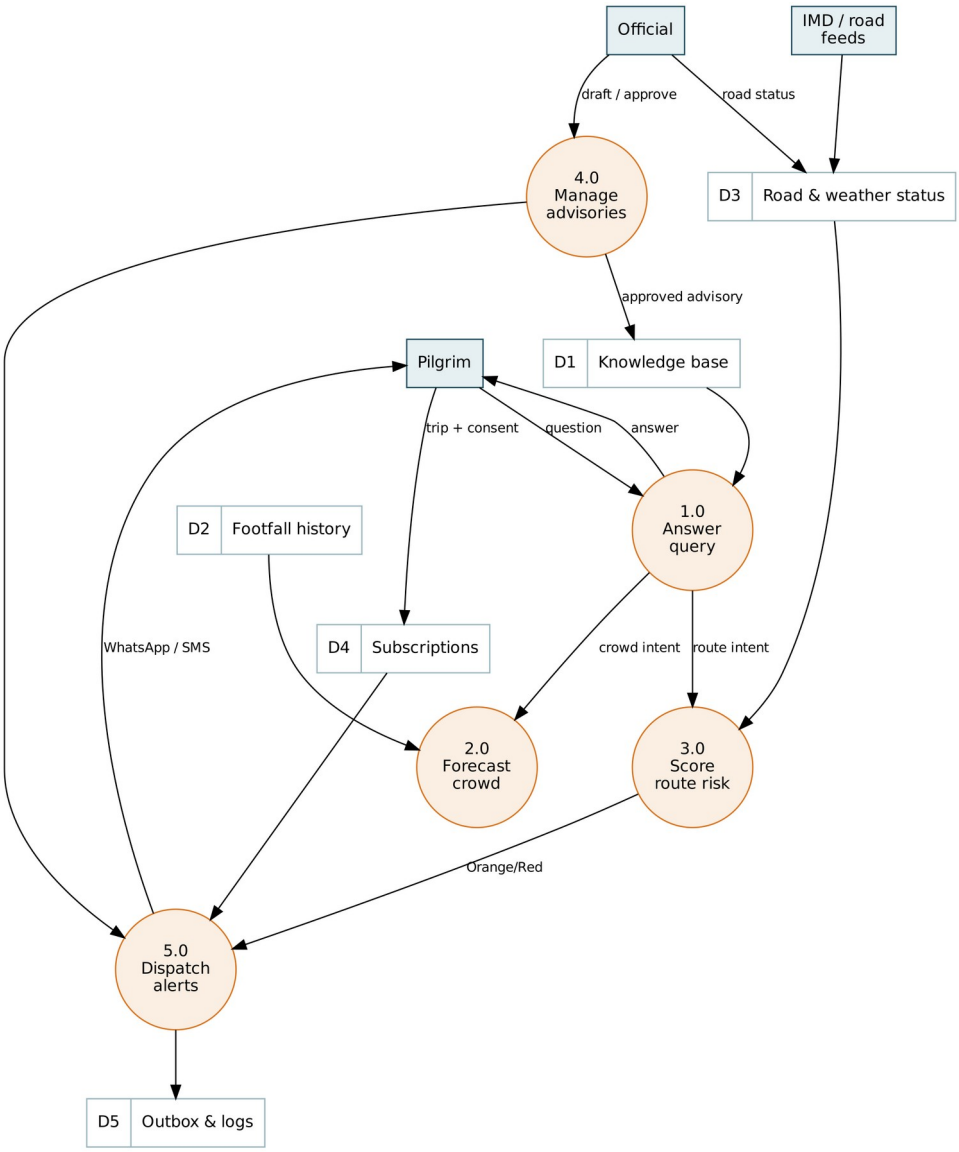


Figure 3.3: Data flow diagram, Level 1

3.8 Entity Relationship Model and Data Dictionary

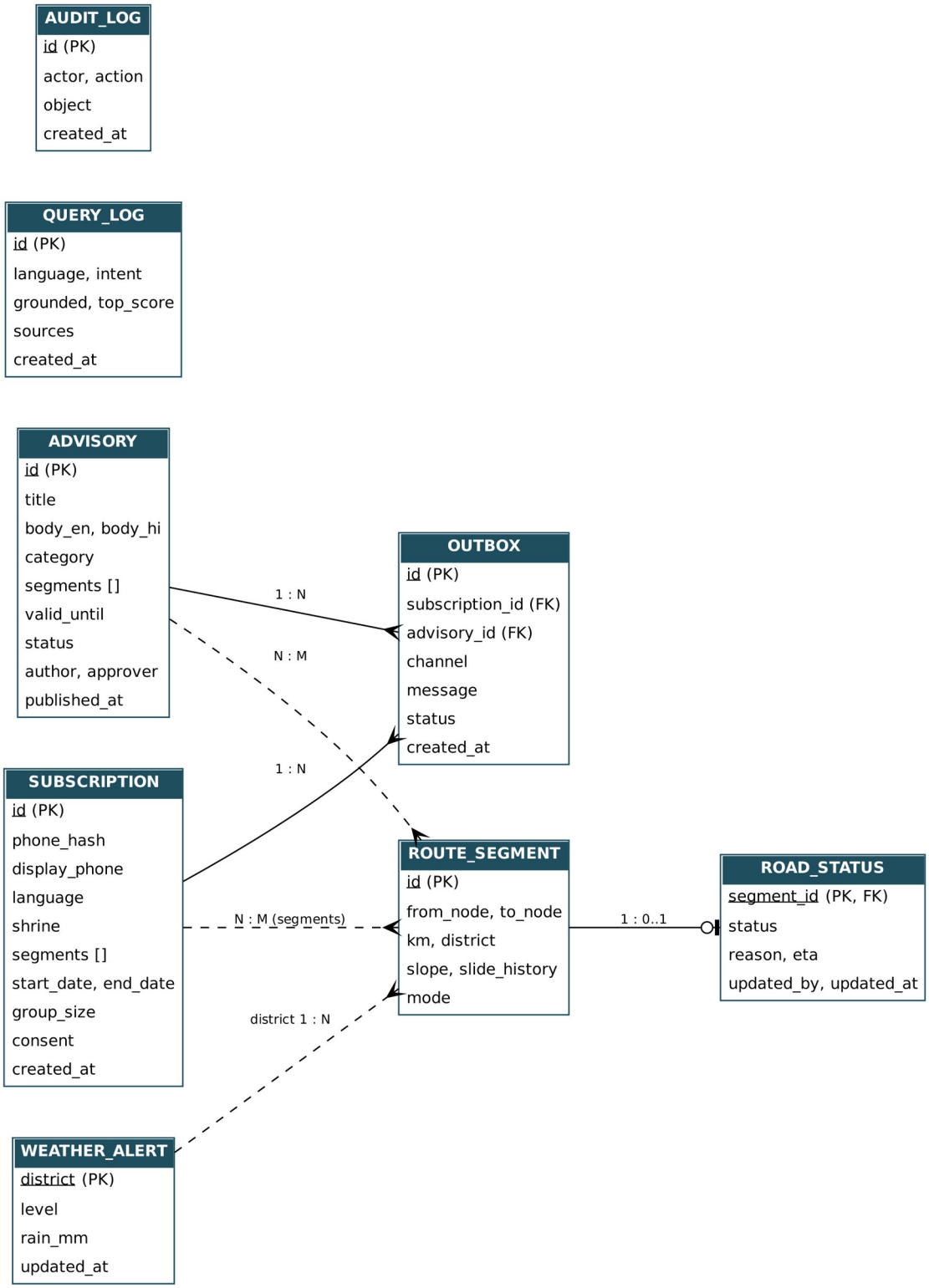


Figure 3.4: Entity relationship diagram

Table 3.6: Data dictionary (main tables)

Table	Key fields	Purpose / retention
subscriptions	id, phone_hash, display_phone, language, shrine, segments, start_date, end_date, group_size, consent	Trips subscribed for alerts; deleted after the trip (90 days in production)
advisories	id, title, body_en, body_hi, category, segments, valid_until, status, author, approver	Official notices; expired advisories are excluded from answers
outbox	id, subscription_id, advisory_id, channel, message, status	Every alert sent, by channel, with delivery status
road_status	segment_id, status, reason, eta, updated_by	Latest official status of each route segment
weather_alerts	district, level, rain_mm	Latest IMD/USDMA alert per district
query_log	language, intent, grounded, top_score, sources	Anonymous analytics; no message text stored
audit_log	actor, action, object, created_at	Accountability for all official actions

3.9 External Interface Requirements

3.9.1 User interfaces

- **Traveller web app:** five tabs (Assistant, Crowd forecast, Route risk, Health check, Trip alerts and SOS); responsive for phones; Devanagari-capable fonts.
- **WhatsApp:** plain-text questions to the business number; answers with source line.
- **SMS:** alert text of up to about 300 characters when WhatsApp delivery fails.
- **Officials' dashboard:** login, overview KPIs and map, advisories, road status and weather, alert delivery, forecast model, audit log.

3.9.2 Software interfaces

Table 3.7: Software interfaces

System	Direction	Data	Protocol
WhatsApp Business Cloud API	In / out	User messages; alerts	HTTPS webhook (verify token), REST
Bhashini pipeline API	Out	Translation for Indian languages	REST (optional)
LLM provider (OpenAI-compatible)	Out	Grounded answer generation	REST (optional)
IMD / USDMA alerts	In	District alert level, rainfall	Feed or dashboard entry

System	Direction	Data	Protocol
Footfall data (tourism department)	In	Daily counts per shrine	CSV (data agreement)
SMS gateway	Out	Fallback alerts	REST (simulated)

3.9.3 Communication interfaces

All client-server communication uses HTTP(S) with JSON bodies. The REST API is self-documented with OpenAPI at the /docs endpoint. Officials authenticate with a session token sent in the X-Token header.

CHAPTER 4
TECHNOLOGY USED AND ITS DESCRIPTION

This chapter describes the architecture of YatraSetu AI and each technology and algorithm used, explaining why it was chosen and how it is applied.

4.1 System Architecture

YatraSetu AI follows a layered, service-oriented architecture. All channels talk to a single API gateway; domain services orchestrate the AI engines; all state is kept in the data layer; and external feeds are read-only.

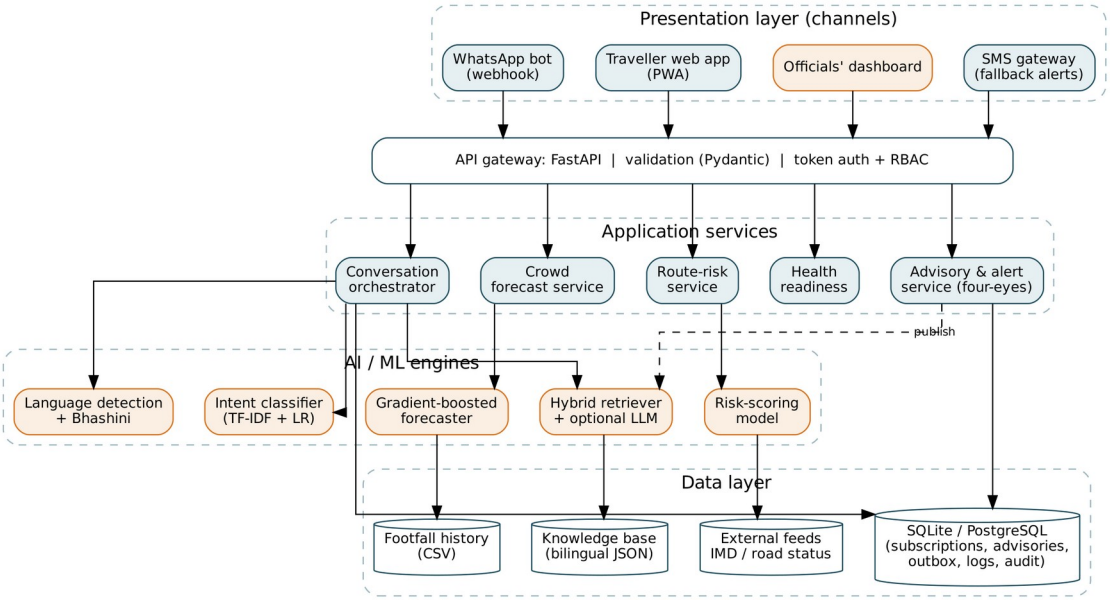


Figure 4.1: Layered architecture of YatraSetu AI

- **Presentation layer:** traveller Progressive Web App, WhatsApp webhook, SMS gateway (outbound) and the officials' dashboard.
- **API gateway:** FastAPI application that validates every request with Pydantic models and enforces token authentication and role-based access for official endpoints.
- **Application services:** conversation orchestrator, crowd-forecast service, route-risk service, health-readiness service and the advisory/alert service.
- **AI/ML engines:** language detection and Bhashini adapter, intent classifier, hybrid retriever with optional LLM, gradient-boosted forecaster and the risk-scoring model.
- **Data layer:** relational database (SQLite in the prototype, PostgreSQL/PostGIS in production), the bilingual knowledge base, the footfall history and external feeds.

4.2 Python and FastAPI

Python was chosen because it is the standard language for machine learning and has mature libraries for data handling and web services. **FastAPI** is a modern, high-performance web framework built on the ASGI standard. Its features used in this project are:

- **Type-based validation with Pydantic:** request bodies such as ChatIn, SubscribeIn and AdvisoryIn are declared as classes with constraints (e.g. phone number pattern, group size between 1 and 60); invalid input is rejected automatically with HTTP 422.
- **Dependency injection:** the functions user() and require(role) are injected into protected endpoints to enforce login and roles (editor vs. approver).
- **Automatic OpenAPI documentation** at /docs for every endpoint, useful for integration with government systems.
- **Uvicorn** ASGI server for serving the API and static files; the app also serves the PWA and dashboard pages.

Table 4.1: Main REST API endpoints

Method and path	Purpose	Access
POST /api/chat	Ask the assistant	Public
GET /api/forecast/{shrine}	7-day crowd forecast	Public
GET /api/forecast/{shrine}/suggest	Lighter dates and alternative shrine	Public
GET /api/risk/network, /api/risk/trip/{shrine}	Route-risk map and trip risk	Public
POST /api/subscribe	Subscribe a trip (consent required)	Public
POST /api/health/check	Health-readiness self-check	Public
POST /api/sos	SOS message and nearest help	Public
GET, POST /webhook/whatsapp	WhatsApp verification and inbound messages	Platform
POST /api/admin/login	Official login, returns token	Officials
POST /api/admin/advisories	Draft advisory	Editor
POST /api/admin/advisories/{id}/approve	Approve and broadcast	Approver
POST /api/admin/road-status, /api/admin/weather	Update road status / IMD alert	Officials
GET /api/admin/analytics, outbox, audit	Dashboard data	Officials
GET /api/model/metrics	Forecast back-test and accuracy	Public (read-only)

4.3 Machine Learning with scikit-learn

scikit-learn (Pedregosa et al., 2011) provides well-tested implementations of the text-representation, classification and regression algorithms used in the project, together with **NumPy** and **pandas** for numerical and tabular data.

4.3.1 TF-IDF representation and cosine similarity

Text is converted to vectors with Term Frequency-Inverse Document Frequency weighting (Salton and Buckley, 1988; Manning et al., 2008). With sublinear term frequency and smoothed inverse document frequency, the weight of term t in document d is:

```
w(t, d) = (1 + ln tf(t, d)) x ( ln((1 + N) / (1 + df(t))) + 1 )

where tf(t,d) = count of t in d,  N = number of documents,  df(t) = documents containing
t.
Vectors are L2-normalised, and similarity is   cos(q, d) = (q . d) / (|q| |d|)
```

Two vectorisers are used together. A **word-level** model rewards exact terms such as "registration" or "Gaurikund". A **character n-gram** model (3 to 5 characters within word boundaries) is robust to spelling variants, Hindi inflections and mixed scripts. The hybrid retrieval score is:

```
score(q, d) = alpha x cos_word(q, d) + (1 - alpha) x cos_char(q, d),   alpha = 0.5

Each document has an English and a Hindi view; the document score is the maximum over its
views.
```

A third, stop-word-filtered word model measures how many *content* words the question shares with the knowledge base. If this content overlap is below a threshold (0.05), the question is treated as off-topic and the assistant gives a safe fallback instead of a loosely related answer (FR-04).

4.3.2 Intent classification with logistic regression

Intent (crowd, route, health, registration, booking, general) is predicted by a multinomial logistic-regression model on character n-gram TF-IDF features, trained on labelled English and Hindi example utterances. For feature vector x and class k the model computes:

```
P(y = k | x) = exp(w_k . x + b_k) / SUM_j exp(w_j . x + b_j)        (softmax)
predicted intent = argmax_k P(y = k | x)
```

A **deterministic emergency override** runs before the model: regular-expression patterns in English and Hindi (e.g. "chest pain", "stuck", "accident", "सीने में दर्द", "फंस")

immediately route the message to the emergency response. This guarantees that emergencies never depend on a probabilistic classifier.

4.3.3 Crowd forecasting with gradient boosting

Footfall is forecast with scikit-learn's **HistGradientBoostingRegressor**, a histogram-based gradient-boosting decision-tree model similar to LightGBM (Ke et al., 2017). Gradient boosting (Friedman, 2001) builds an additive model of regression trees, each fitted to the negative gradient (residuals) of the loss of the previous model:

```
F_0(x) = mean(y)
F_m(x) = F_(m-1)(x) + nu x h_m(x),   h_m fitted to residuals r_i = y_i - F_(m-1)(x_i)

Target: y = ln(1 + visitors)   (stabilises variance across shrines)
Settings: 400 iterations, learning rate nu = 0.05, 31 leaves, shrine as categorical
feature
```

Table 4.2: Forecast model features (all known 7 days in advance)

Feature	Description
shrine_code	Shrine as a categorical feature
day_of_season, days_to_close	Position in the season (captures opening surge and closing tail)
month, weekday, is_weekend	Calendar effects
is_holiday	National holidays and summer-vacation period
rain_mm, alert_level	Forecast rainfall and IMD alert level (climatology when no forecast is available)

Accuracy is measured with the Mean Absolute Percentage Error on a held-out season, excluding disrupted days (closures), and with the share of days whose predicted crowd band matches the actual band:

```
MAPE = (100 / n) x SUM_i | y_i - y_hat_i | / y_i

Crowd band from ratio r = forecast / capacity:
  Low: r < 0.60    Moderate: 0.60 <= r < 0.85    High: 0.85 <= r < 1.00    Over capacity:
r >= 1.00
```

Because official shrine-wise daily counts require a data-sharing agreement, the model is trained on a **synthetic history** for 2022 to 2025 generated by a documented script. The generator reproduces the observed shape of the yatra: season totals calibrated to published figures (about 51 lakh in 2025), an opening-day surge (about 31,000 at Kedarnath, close to the reported 30,154), a May-June peak, a monsoon trough with

closure days, and weekend/holiday uplift. The pipeline is unchanged when real data replaces the file.

4.4 Retrieval-Augmented Generation, LLM Integration and Bhashini

The assistant follows the RAG pattern (Lewis et al., 2020). The flow of a message is shown below.

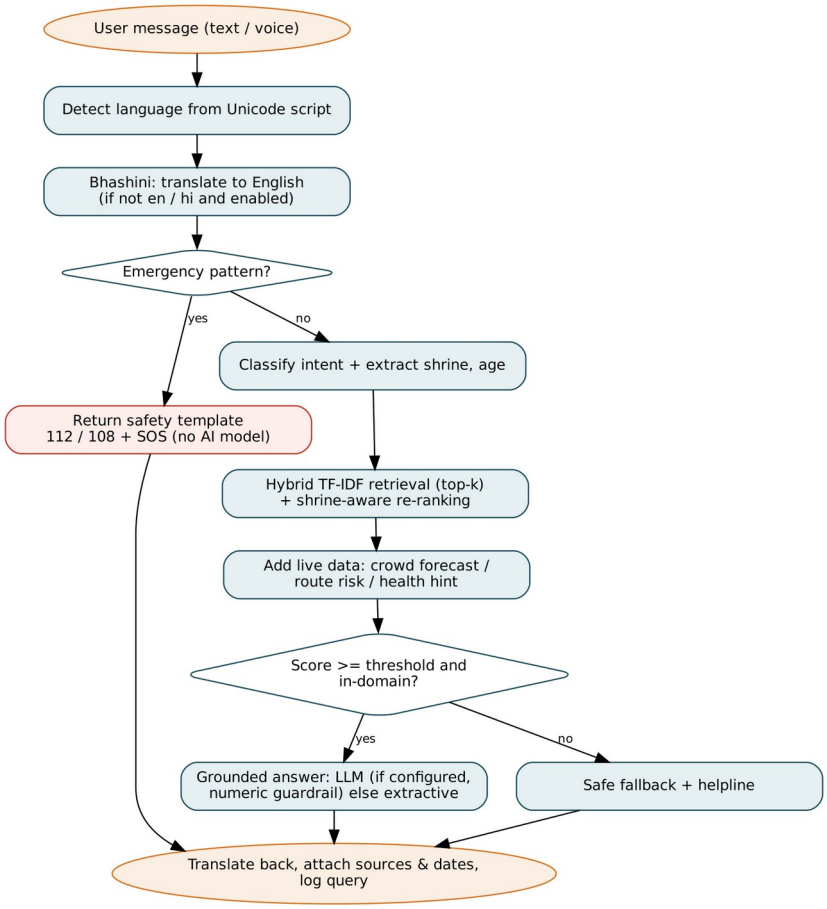


Figure 4.2: Processing flow of a user message in the assistant

1. **Language detection:** the dominant Unicode script of the message identifies the language (Devanagari, Gujarati, Tamil, etc.; Latin defaults to English).
2. **Translation (optional):** for languages other than English and Hindi, the Bhashini pipeline translates to English and the answer back; without credentials the system answers in English with a note.
3. **Emergency check and intent classification** (Section 4.3.2) and extraction of the shrine name and age.
4. **Retrieval:** hybrid TF-IDF search over the bilingual knowledge base and published advisories, with shrine-aware re-ranking (documents whose title names the requested shrine are boosted).

5. **Live enrichment:** for crowd questions the 7-day forecast is added; for route questions the trip risk and safe halt are added.
6. **Answer generation:** if an LLM is configured, the top passages are sent with a strict system prompt (answer only from passages, cite them, say NOT_FOUND otherwise). A guardrail checks that every number in the answer appears in the passages (reducing hallucination; Ji et al., 2023); otherwise, or if no LLM is configured, an **extractive** answer is built from the most relevant sentences of the best passage.
7. **Output:** sources and last-updated dates are attached and an anonymous log entry (language, intent, grounded flag, sources) is stored.

Before any text is sent to an external LLM, phone numbers and e-mail addresses are redacted. If the LLM or Bhashini is unavailable, the system degrades gracefully to the offline extractive mode.

4.5 Route-Risk Scoring Algorithm

Following the principle of combining dynamic triggers with static susceptibility (Kirschbaum and Stanley, 2018), each of the 18 route segments receives a transparent 0-100 score:

```
R = A + 20 x min(rain_mm, 150) / 150 + w x S + D            (capped at 100)

A = alert points:        None 0, Yellow 15, Orange 30, Red 45   (IMD district alert)
S = susceptibility:      5 x (slope_class - 1) + 1.5 x min(past_slides, 10)
w = 1.0 if an alert is active or rain >= 20 mm, else 0.4
D = road-status points:  Open 0, Restricted 20, Closed 100

Level: R < 25 Green,  R < 45 Yellow,  R < 65 Orange,  otherwise Red;  Closed is always
Red
```

Every contributing factor is returned with the score and shown to the user (e.g. "IMD Orange alert in Rudraprayag (+30)"), so the advice is explainable. When the worst segment of a trip is Orange or Red, the **safe-halt** search walks back along the trip to the nearest town that has both accommodation and medical facilities.

For SOS, the nearest town with medical facilities is found with the haversine great-circle distance between the user's GPS position and town coordinates:

```
a = sin^2(dlat/2) + cos(lat1) cos(lat2) sin^2(dlon/2)
distance = 2 x 6371 km x asin( sqrt(a) )
```

4.6 Health-Readiness Rules

The health module is deliberately rule-based and conservative, because medical advice must be predictable and must not be presented as a diagnosis.

Table 4.3: Health-readiness rules

Input	Rule	Result
Any red flag (chest pain, breathlessness at rest, recent surgery/heart attack, fainting)	Overrides everything	Do not trek now + 112/108
Age 60+ / 50 to 59	+2 / +1 points	Screening reminder for 50+
Heart disease, lung disease, pregnancy	+2 points each	
High BP, diabetes	+1 point each	
Low walking fitness	+1 point	
Total points	0 / 1-2 / 3 or more	Ready / Caution / Consult doctor first

4.7 Database

SQLite is used in the prototype because it needs no server and is part of Python. The schema (Section 3.8) is portable to **PostgreSQL** with **PostGIS** for production, where segment geometries and spatial queries (e.g. subscribers within a buffer around a landslide) can be handled natively. Phone numbers are never stored in clear text: a salted SHA-256 hash is used to identify the subscriber and only a masked form (e.g. 98******10) is shown to officials.

4.8 Front-End: HTML5, CSS3, JavaScript and Progressive Web App

The traveller app and dashboard are written in plain HTML5, CSS3 and JavaScript without external libraries or CDNs, so they load quickly on weak networks. Charts are drawn on the HTML **Canvas** and the route-risk map is generated as **SVG** from the route network. A web-app **manifest** and a **service worker** make the app installable and cache its assets for offline use. The layout is responsive and uses fonts that support Devanagari.

4.9 Messaging and Advisory Workflow

Inbound WhatsApp messages arrive at a webhook in the format of the WhatsApp Business Cloud API and are answered by the same assistant. Outbound alerts are written to an **outbox** table, which simulates the WhatsApp and SMS gateways: if WhatsApp delivery fails, an SMS copy is sent. The advisory workflow enforces the four-eyes principle, as shown in the sequence diagram.

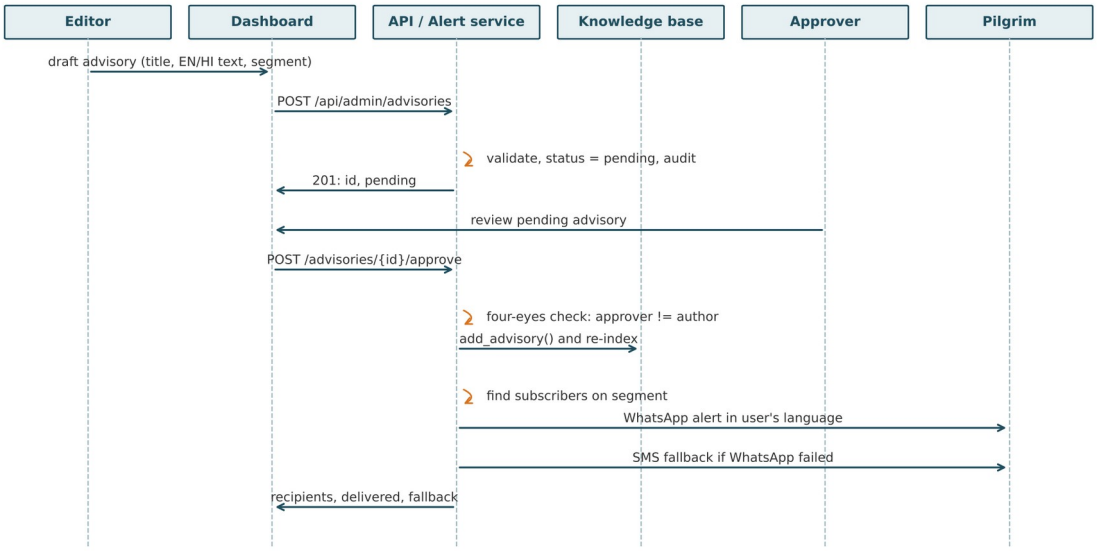


Figure 4.3: Sequence diagram of advisory drafting, approval and broadcast

4.10 Security and Privacy

- **Authentication and RBAC:** officials log in and receive a random session token; editor and approver roles are enforced on each endpoint.
- **Four-eyes rule:** an advisory cannot be approved by its author.
- **Input validation:** every request body is validated with Pydantic (types, lengths, patterns).
- **Data minimisation and consent:** trip details are stored only with explicit consent; analytics store intent and language, not message text (DPDP Act, 2023).
- **PII redaction:** phone numbers and e-mails are removed before text is sent to an external LLM.
- **Audit log:** logins, advisories, approvals, road and weather changes and SOS events are recorded with actor and time.

4.11 Testing Tools

pytest runs 18 automated unit and API tests, using FastAPI's **TestClient** to call the endpoints in-process. A separate evaluation script measures retrieval, intent, emergency and forecast accuracy on held-out data. **Playwright** (headless Chromium) was used to drive the user interface and capture the screenshots in Chapter 6.

CHAPTER 5
CODING

The prototype is written in about 2,400 lines of Python, JavaScript, HTML and CSS. This chapter presents the project structure and the most important parts of the code. The complete, runnable source code is provided in the project folder **YatraSetu-AI** and can be started with the single command ./run.sh.

5.1 Project Structure

```
YatraSetu-AI/
├── app/
│   ├── main.py             FastAPI app: REST API, WhatsApp webhook, pages
│   ├── assistant.py        conversation orchestrator
│   ├── knowledge_base.py   bilingual knowledge base + hybrid retriever
│   ├── intent.py           intent classifier + emergency override
│   ├── language.py         language detection + Bhashini client
│   ├── llm.py              optional LLM call + grounding guardrail
│   ├── forecast.py         crowd forecasting model
│   ├── risk.py             route-risk scoring, safe halt
│   ├── health.py           health-readiness rules
│   ├── alerts.py           subscriptions, advisories, alert dispatch
│   ├── db.py, config.py    persistence and configuration
│   ├── data/               knowledge_base.json, shrines.json,
│   │                       routes.json, footfall_history.csv
│   └── static/             index.html, app.js, admin.html, admin.js,
│                           charts.js, style.css, manifest.json, sw.js
├── scripts/                generate_data.py, seed_demo.py, evaluate.py
├── tests/                  test_core.py, gold_set.json
├── docs/                   screenshots/, evaluation_results.json
├── requirements.txt, run.sh, README.md
```

Table 5.1: Source modules

Module	Lines	Responsibility
app/main.py	292	API gateway, request models, authentication, WhatsApp webhook
app/assistant.py	158	Conversation orchestrator
app/alerts.py	133	Subscriptions, advisories, alert dispatch
app/forecast.py	126	Crowd forecasting and evaluation
app/knowledge_base.py	114	Knowledge base and hybrid retriever
app/intent.py	105	Intent classifier, emergency override, entity extraction
app/db.py	105	Database schema and helpers
app/risk.py	103	Route-risk scoring and safe-halt search
app/language.py	86	Language detection, Bhashini client
app/health.py, app/llm.py, app/config.py	141	Health rules, LLM guardrail, configuration

Module	Lines	Responsibility
app/static/*	673	Traveller PWA, dashboard, charts, styles, service worker
scripts/*	203	Synthetic data generation, demo seeding, evaluation
tests/test_core.py	137	18 unit and API tests

5.2 Hybrid TF-IDF retriever (app/knowledge_base.py)

The retriever builds three TF-IDF models over the English and Hindi views of each document. search() combines word and character similarities and keeps the best view per document; content_overlap() is the off-topic gate.

```
    def build(self):
        self.views = []  # (doc_index, lang, text)
        active = self.active_docs()
        self._active = active
        for i, d in enumerate(active):
            for lang, t in d.text.items():
                self.views.append((i, lang, f"{d.title}. {t}"))
        corpus = [v[2] for v in self.views]
        self.word_vec = TfidfVectorizer(analyzer="word", token_pattern=r"(?u)\b\w+\b",
sublinear_tf=True)
        # content-word model (stop words removed) used as an out-of-domain gate (FR-04)
        self.gate_vec = TfidfVectorizer(analyzer="word", token_pattern=TOKEN,
stop_words=STOP, sublinear_tf=True)
        self.Xg = self.gate_vec.fit_transform(corpus)
        self.char_vec = TfidfVectorizer(analyzer="char_wb", ngram_range=(3, 5),
sublinear_tf=True)
        self.Xw = self.word_vec.fit_transform(corpus)
        self.Xc = self.char_vec.fit_transform(corpus)

    def search(self, query: str, k: int = 3, alpha: float = 0.5):
        """Return [(doc, lang_of_best_view, score)] sorted by score."""
        sw = cosine_similarity(self.word_vec.transform([query]), self.Xw)[0]
        sc = cosine_similarity(self.char_vec.transform([query]), self.Xc)[0]
        s = alpha * sw + (1 - alpha) * sc
        best = {}
        for vi, score in enumerate(s):
            di, lang, _ = self.views[vi]
            if di not in best or score > best[di][1]:
                best[di] = (lang, float(score))
        ranked = sorted(best.items(), key=lambda kv: kv[1][1], reverse=True)[:k]
        return [(self._active[di], lang, score) for di, (lang, score) in ranked]

    def content_overlap(self, query: str) -> float:
        """Max similarity on content words only; ~0 for questions unrelated to the
yatra."""
        return float(cosine_similarity(self.gate_vec.transform([query]), self.Xg).max())
```

5.3 Intent classifier with emergency override (app/intent.py)

The classifier is a scikit-learn pipeline of character n-gram TF-IDF and multinomial logistic regression. classify() first applies the deterministic emergency patterns.

```
    def __init__(self):
        X, y = [], []
        for label, examples in TRAINING.items():
            X += examples
            y += [label] * len(examples)
        self.model = make_pipeline(
            TfidfVectorizer(analyzer="char_wb", ngram_range=(2, 5), sublinear_tf=True),
            LogisticRegression(max_iter=2000, C=8.0),
        )
        self.model.fit(X, y)
        shrines = json.loads((DATA_DIR / "shrines.json").read_text(encoding="utf-8"))
["shrines"]
        self.shrine_terms = {}
        for s in shrines:
            for term in {s["id"], s["name"].lower(), s["name_hi"], s["name"].split()
[0].lower()}:
                self.shrine_terms[term] = s["id"]
        self.shrine_terms.update({"kedar": "kedarnath", "badri": "badrinath", "hemkund":
"hemkund",
                                  "केदार": "kedarnath", "बद्री": "badrinath", "बदरीनाथ":
"badrinath"})

    def classify(self, text: str):
        if self.is_emergency(text):
            return "emergency", 1.0
        proba = self.model.predict_proba([text.lower()])[0]
        i = proba.argmax()
        return self.model.classes_[i], float(proba[i])
```

5.4 Conversation orchestrator (app/assistant.py)

respond() implements the full pipeline of Figure 4.2: language handling, emergency path, retrieval with shrine-aware re-ranking, live enrichment, grounded answer generation and logging.

```
def respond(text: str, lang_override: str | None = None):
    lang = lang_override or detect_language(text)
    work_lang, note = lang, ""
    query = text
    if lang not in ("en", "hi"):
        if bhashini.enabled:
            query = bhashini.translate(text, lang, "en")
            work_lang = "en"
        else:
            work_lang = "en"
            note = T["en"]["other_lang"].format(lang=LANG_NAMES.get(lang, lang))
    intent, conf = classifier.classify(query)
    shrine = classifier.shrine(query)
    result = {"language": lang, "intent": intent, "confidence": round(conf, 2), "shrine":
shrine,
              "sources": [], "live": {}, "grounded": False, "mode": "extractive"}
```

```
    if intent == "emergency":                      # FR-04: deterministic, never via the
LLM
        result.update(answer=T[work_lang]["emergency"], grounded=True, mode="safety-
template", sos=True)
        return _finish(result, lang, work_lang, note, 1.0)

    hits = kb.search(query, k=6)
    if shrine:  # shrine-aware re-ranking: prefer passages that mention the shrine the
user asked about
        names = {forecaster.shrines[shrine]["name"], forecaster.shrines[shrine]
["name_hi"]}
        def boost(d):
            if any(n in d.title for n in names):
                return 1.8
            return 1.1 if any(n in " ".join(d.text.values()) for n in names) else 1.0
        hits = sorted(((d, l, sc * boost(d)) for d, l, sc in hits), key=lambda h: -h[2])
    hits = hits[:TOP_K] if not (intent in INTENT_TOPICS) else hits
    top = hits[0][2] if hits else 0.0
    parts = []
    if intent == "crowd" and shrine:
        line, fc = _crowd_line(shrine, work_lang)
        parts.append(line); result["live"]["forecast"] = fc
    if intent == "route" and shrine:
        line, tr = _route_line(shrine, work_lang)
        parts.append(line); result["live"]["route"] = {"worst": tr["worst"], "advice":
tr["advice"]}
    age = classifier.age(query)
    if intent == "health" and age and age >= 50:
        parts.append(T[work_lang]["health_age"].format(age=age))

    in_domain = kb.content_overlap(query) >= CONTENT_THRESHOLD
    if parts:
        # live data already answers the question: only add knowledge that matches the
intent's topic
        topics = INTENT_TOPICS.get(intent, set())
        hits = [h for h in hits if h[0].topic in topics]
        top = hits[0][2] if hits else 0.0
    hits = hits[:TOP_K]
    if top >= RETRIEVAL_THRESHOLD and in_domain:
        doc, view_lang, _ = hits[0]
        passages = [h[0].text.get(work_lang) or h[0].text["en"] for h in hits if h[2] >=
max(RETRIEVAL_THRESHOLD, 0.7 * top)]
        generated = llm.generate(query, passages)
        if generated:
            parts.append(generated); result["mode"] = "llm-rag"
        else:
            use_lang = work_lang if work_lang in doc.text else "en"
            parts.append(kb.best_sentences(doc, use_lang, query, n=3))
        result["grounded"] = True
        result["sources"] = [{"title": h[0].title, "source": h[0].source, "updated":
h[0].updated, "score": round(h[2], 3)}
                             for h in hits if h[2] >= max(RETRIEVAL_THRESHOLD, 0.7 *
top)]
    elif parts:
        result["grounded"] = True                # answer built only from live, official
data feeds
        result["sources"] = [{"title": "Live data", "source": "YatraSetu forecast /
route-risk services",
                              "updated": date.today().isoformat(), "score": None}]
    else:
        parts.append(T[work_lang]["fallback"]); result["mode"] = "fallback"
```

```
    result["answer"] = " ".join(parts)
    return _finish(result, lang, work_lang, note, top)
```

5.5 Crowd forecasting model and evaluation (app/forecast.py)

The model is trained on log(1 + visitors). _evaluate() trains on earlier seasons and tests on the latest one, reporting MAPE on normal days and crowd-band accuracy.

```
    def _fit(self, df):
        m = HistGradientBoostingRegressor(max_iter=400, learning_rate=0.05,
max_leaf_nodes=31,
                                          categorical_features=[0], random_state=7)
        m.fit(df[FEATURES], np.log1p(df["visitors"]))
        return m

    def _evaluate(self):
        """Train on all seasons except the last, test on the last (NFR-13)."""
        test_year = int(self.df["year"].max())
        train, test = self.df[self.df.year < test_year], self.df[self.df.year ==
test_year]
        m = self._fit(train)
        pred = np.expm1(m.predict(test[FEATURES]))
        normal = (test["disrupted"] == 0) & (test["visitors"] >= 200)
        per_shrine = {}
        for sid in self.shrines:
            mask = normal & (test["shrine"] == sid)
            per_shrine[sid] = round(mape(test.loc[mask, "visitors"], pred[mask.values]),
1)
        # Band accuracy: does the predicted crowd band match the actual band?
        cap = test["shrine"].map(lambda s: self.shrines[s]["capacity"])
        band_ok = [band(p / c) == band(a / c) for p, a, c in zip(pred, test["visitors"],
cap)]
        return {
            "test_season": test_year, "train_rows": int(len(train)), "test_rows":
int(len(test)),
            "mape_normal_days": round(mape(test.loc[normal, "visitors"],
pred[normal.values]), 1),
            "mape_by_shrine": per_shrine,
            "band_accuracy": round(100 * float(np.mean(band_ok)), 1),
            "test_frame": test.assign(predicted=pred.round()),
        }

    def forecast(self, sid: str, start: date, days: int = 7, weather=None):
        """weather: optional {date_iso: (rain_mm, alert_level)} from the live IMD
feed."""
        weather = weather or {}
        cap = self.shrines[sid]["capacity"]
        out = []
        for i in range(days):
            d = start + timedelta(i)
            rain, lvl = weather.get(d.isoformat(), (None, 0))
            feats, open_ = self.features_for(sid, d, rain, lvl)
            if not open_:
                out.append({"date": d.isoformat(), "open": False, "visitors": 0, "ratio":
0, "band": "Closed"})
                continue
            v = float(np.expm1(self.model.predict(pd.DataFrame([feats])[FEATURES])[0]))
            out.append({"date": d.isoformat(), "open": True, "visitors": int(round(v, -
1)),
```

```
                        "ratio": round(v / cap, 2), "band": band(v / cap), "alert_level":
lvl})
        return {"shrine": sid, "name": self.shrines[sid]["name"], "capacity": cap,
"days": out}
```

5.6 Route-risk scoring and safe-halt search (app/risk.py)

score_segment() is a pure function implementing the formula of Section 4.5 and returning human-readable factors. safe_halt() walks back along the trip to a town with stay and medical facilities.

```
def score_segment(seg: dict, alert: str = "None", rain_mm: float = 0.0, status: str =
"Open"):
    """Return (score, level, factors). Pure function, unit-tested."""
    factors = []
    s = 0.0
    a = ALERT_POINTS.get(alert, 0)
    if a:
        factors.append(f"IMD {alert} alert in {seg['district']} (+{a})")
    s += a
    r = min(rain_mm, 150) / 150 * 20                    # up to +20 for very heavy rain
    if r >= 1:
        factors.append(f"Forecast rain {rain_mm:.0f} mm (+{r:.0f})")
    s += r
    susc = (seg["slope"] - 1) * 5 + min(seg["slide_history"], 10) * 1.5   # up to +25
    # susceptibility only matters when it is raining or an alert is active
    weight = 1.0 if (a or rain_mm >= 20) else 0.4
    s += susc * weight
    factors.append(f"Terrain susceptibility (slope class {seg['slope']},
{seg['slide_history']} past slides) (+{susc * weight:.0f})")
    st = STATUS_POINTS.get(status, 0)
    if st:
        factors.append(f"Road status: {status} (+{st})")
    s += st
    s = min(round(s), 100)
    lvl = "Red" if status == "Closed" else level(s)
    return s, lvl, factors

def safe_halt(segment_id: str):
    """FR-17: the nearest town with stay + medical facilities on the traveller's side of
the blockage."""
    seg = SEGMENTS[segment_id]
    # walk back along any trip that contains this segment
    for trip in TRIPS.values():
        if segment_id in trip:
            idx = trip.index(segment_id)
            for back in reversed(trip[: idx + 1]):
                node = SEGMENTS[back]["from"]
                f = NODES[node]["facilities"]
                if "stay" in f and ("medical" in f or "hospital" in f):
                    return {"node": node, "name": NODES[node]["name"], "facilities": f}
    n = NODES[seg["from"]]
    return {"node": seg["from"], "name": n["name"], "facilities": n["facilities"]}
```

5.7 Four-eyes approval and alert dispatch (app/alerts.py)

approve_advisory() enforces the four-eyes rule, adds the advisory to the knowledge base and dispatches alerts. _send() simulates the WhatsApp gateway with SMS fallback.

```
def _send(sub, advisory_id, text):
    """Simulated gateway: numbers whose masked form ends in '0' fail on WhatsApp to demo
SMS fallback."""
    ok = not sub["display_phone"].endswith("0")
    db.execute("INSERT INTO outbox(subscription_id, advisory_id, channel, message,
status, created_at) VALUES (?,?,?,?,?,?)",
               (sub["id"], advisory_id, "whatsapp", text, "delivered" if ok else
"failed", db.now()))
    if not ok:  # FR-16: SMS fallback
        db.execute("INSERT INTO outbox(subscription_id, advisory_id, channel, message,
status, created_at) VALUES (?,?,?,?,?,?)",
                   (sub["id"], advisory_id, "sms", text[:300], "delivered", db.now()))
    return ok

def approve_advisory(aid: int, approver: str):
    adv = db.rows("SELECT * FROM advisories WHERE id=?", (aid,))
    if not adv:
        raise ValueError("Advisory not found")
    adv = adv[0]
    if adv["status"] != "pending":
        raise ValueError("Advisory already processed")
    if adv["author"] == approver:
        raise PermissionError("Four-eyes rule: the author cannot approve their own
advisory")
    db.execute("UPDATE advisories SET status='published', approver=?, published_at=?
WHERE id=?", (approver, db.now(), aid))
    adv = db.rows("SELECT * FROM advisories WHERE id=?", (aid,))[0]
    kb.add_advisory(adv)                                  # FR-25: knowledge base updated
    sent = failed = 0
    for sub in _affected(json.loads(adv["segments"])):    # FR-25: broadcast to affected
subscribers
        if _send(sub, aid, compose(adv, sub)):
            sent += 1
        else:
            failed += 1
    db.audit(approver, "publish_advisory", {"id": aid, "whatsapp_delivered": sent,
"sms_fallback": failed})
    return {"id": aid, "recipients": sent + failed, "whatsapp_delivered": sent,
"sms_fallback": failed}
```

5.8 LLM grounding guardrail (app/llm.py)

Text is redacted before leaving the system, and a generated answer is rejected if it contains a number that does not appear in the retrieved passages.

```
def redact(text: str) -> str:
    """Remove phone numbers and e-mail addresses before text leaves the system (NFR-
16)."""
    text = re.sub(r"\b\d{10,12}\b", "[PHONE]", text)
    return re.sub(r"\S+@\S+", "[EMAIL]", text)
```

```
def numbers_grounded(answer: str, passages: list[str]) -> bool:
    nums = set(re.findall(r"\d[\d,\.]*", answer)) - {str(i) for i in range(1, 10)}
    joined = " ".join(passages)
    return all(n.rstrip(".,") in joined for n in nums)

def generate(question: str, passages: list[str]):
    if not enabled():
        return None
    import requests
    ctx = "\n".join(f"[{i + 1}] {p}" for i, p in enumerate(passages))
    body = {"model": LLM_MODEL, "temperature": 0.1, "max_tokens": 300,
            "messages": [{"role": "system", "content": SYSTEM_PROMPT},
                         {"role": "user", "content": f"Passages:\n{ctx}\n\nQuestion:
{redact(question)}"}]}
    try:
        r = requests.post(LLM_API_URL, json=body, timeout=20, headers={"Authorization":
f"Bearer {LLM_API_KEY}"})
        r.raise_for_status()
        ans = r.json()["choices"][0]["message"]["content"].strip()
    except Exception:
        return None                       # NFR-05 graceful degradation
    if "NOT_FOUND" in ans or not numbers_grounded(ans, passages):
        return None
    return ans
```

5.9 Health-readiness rules (app/health.py)

assess() applies the rules of Table 4.3 and lists screening points and medical posts along the chosen route.

```
def assess(age: int, conditions: list[str], red_flags: list[str], fitness: str, shrine:
str):
    """Return category (Ready / Caution / Consult doctor first / Do not trek now) with
reasons and actions."""
    reasons, actions = [], []
    if red_flags:
        cat = "Do not trek now"
        reasons += [RED_FLAGS[r] for r in red_flags if r in RED_FLAGS]
        actions += ["Do not start the trek. Visit the nearest medical relief post or
hospital today.",
                    "In an emergency call 112 (or 108 for an ambulance)."]
    else:
        points = 0
        if age >= 60:
            points += 2; reasons.append("Age 60 or above")
        elif age >= 50:
            points += 1; reasons.append("Age 50 or above")
        for c in conditions:
            if c in CONDITIONS:
                points += 2 if c in ("heart", "asthma", "pregnancy") else 1
                reasons.append(CONDITIONS[c])
        if fitness == "low":
            points += 1; reasons.append("Low walking fitness (cannot walk 5 km
comfortably)")
        cat = "Ready" if points == 0 else "Caution" if points <= 2 else "Consult doctor
first"
        if cat == "Consult doctor first":
            actions.append("Get a doctor's fitness certificate before travelling;
consider pony, palki or helicopter instead of walking.")
```

```
        if cat != "Ready":
            actions.append("Rest a night at an intermediate halt to acclimatise; walk
slowly with breaks.")
        actions.append("Carry your regular medicines and prescriptions; drink enough
water; keep warm.")
    if age >= 50:
        actions.insert(0, "Health screening is mandatory for pilgrims aged 50+: get
screened at the screening point on your route.")
    # FR-19 / FR-20: screening points and medical posts along the chosen route
    nodes = []
    for sid in TRIPS.get(shrine, []):
        for n in (SEGMENTS[sid]["from"], SEGMENTS[sid]["to"]):
            if n not in nodes:
                nodes.append(n)
    screening = [NODES[n]["name"] for n in nodes if "screening" in NODES[n]
["facilities"]]
    medical = [NODES[n]["name"] for n in nodes if {"medical", "hospital"} & set(NODES[n]
["facilities"])]
    return {"category": cat, "reasons": reasons, "actions": actions,
            "screening_points": screening, "medical_posts": medical,
            "disclaimer": "This is general readiness guidance, not a medical diagnosis.
Follow doctors and official health advisories."}
```

5.10 REST API endpoints and role-based access (app/main.py)

require() is a FastAPI dependency factory enforcing roles. The chat, approve and SOS endpoints show validation, role checks and the haversine nearest-help search.

```
def require(role):
    def dep(u=Depends(user)):
        if u["role"] != role:
            raise HTTPException(403, f"Only {role}s can do this")
        return u
    return dep

@app.post("/api/chat")
def chat(body: ChatIn):
    return respond(body.text.strip(), body.language)

@app.post("/api/admin/advisories/{aid}/approve")
def approve(aid: int, u=Depends(require("approver"))):
    try:
        return alerts.approve_advisory(aid, u["username"])
    except (ValueError, PermissionError) as e:
        raise HTTPException(400, str(e))

@app.post("/api/sos")
def sos(body: SosIn):
    maps = f"https://maps.google.com/?q={body.lat:.5f},{body.lon:.5f}"
    msg = f"SOS from {body.name} via YatraSetu. Location: {maps}. Please call me or 112."
    help_nodes = [n for n in NODES.values() if {"medical", "hospital"} &
set(n["facilities"])]
    nearest = min(help_nodes, key=lambda n: haversine_km(body.lat, body.lon, n["lat"],
n["lon"]))
    dist = haversine_km(body.lat, body.lon, nearest["lat"], nearest["lon"])
    db.audit("public", "sos", {"lat": body.lat, "lon": body.lon})
    return {"message": msg, "dial": "tel:112", "whatsapp_share": f"https://wa.me/?
text={quote(msg)}",
            "sms_links": [f"sms:{c}?body={quote(msg)}" for c in body.contacts],
```

```
"nearest_help": nearest["name"], "nearest_km": round(dist, 1),
            "nearest_facilities": nearest["facilities"]}
```

5.11 Chat client in the traveller web app (app/static/app.js)

The browser client posts the question, renders the answer with its sources and fills the 'last response details' panel.

```
async function ask(text) {
  if (!text.trim()) return;
  addMsg(esc(text), "user"); $("#q").value = "";
  const wait = addMsg("…", "bot");
  try {
    const r = await api("/api/chat", { text, language: $("#lang").value || null });
    const src = r.sources.map(s => `<span class="src">📄 ${esc(s.title)} · $
{esc(s.updated)}</span>`).join("");
    wait.innerHTML = esc(r.answer) + (src ? `<div class="meta">${src}</div>` : "");
    if (r.sos) { wait.classList.add("sos"); wait.innerHTML += `<div class="meta"><a
href="tel:112"><b>📞 Call 112</b></a></div>`; }
    $("#details").innerHTML = `<table class="t"><tr><th>Field</th><th>Value</th></tr>
      <tr><td>Language</td><td>${esc(r.language)}</td></tr><tr><td>Intent</td><td>$
{esc(r.intent)} (confidence ${r.confidence})</td></tr>
      <tr><td>Shrine detected</td><td>${esc(r.shrine || "–")}</td></tr><tr><td>Answer
mode</td><td>${esc(r.mode)}</td></tr>
      <tr><td>Grounded</td><td>${r.grounded ? "✅ yes" : "⚠️ no (fallback)"}</td></tr>
      <tr><td>Sources (similarity)</td><td>${r.sources.map(s => esc(s.title) + (s.score !
= null ? ` (${s.score})` : "")).join("<br>") || "–"}</td></tr></table>`;
  } catch (e) { wait.innerHTML = "Sorry, something went wrong: " + esc(e.message); }
}
```

5.12 Integration test of the advisory workflow (tests/test_core.py)

This integration test subscribes two travellers, checks that roles are enforced, approves an advisory, verifies the SMS fallback and confirms that the assistant can immediately answer from the new advisory.

```
def test_four_eyes_and_broadcast_with_sms_fallback():
    db.init_db()
    today = date.today().isoformat()
    alerts.subscribe("9876543210", "hi", "kedarnath", today, today, 2, True)   # masked
number ends in 0 -> SMS fallback
    alerts.subscribe("9811122233", "en", "kedarnath", today, today, 2, True)
    ed = client.post("/api/admin/login", json={"username": "editor", "password":
"editor123"}).json()["token"]
    ap = client.post("/api/admin/login", json={"username": "approver", "password":
"approver123"}).json()["token"]
    adv = {"title": "Road closed", "body_en": "Debris near Sonprayag.", "body_hi":
"सोनप्रयाग के पास मलबा।",
           "segments": ["guptkashi_sonprayag"], "valid_until": today}
    # approver cannot draft; editor cannot approve
    assert client.post("/api/admin/advisories", json=adv, headers={"X-Token":
ap}).status_code == 403
    aid = client.post("/api/admin/advisories", json=adv, headers={"X-Token": ed}).json()
["id"]
    assert client.post(f"/api/admin/advisories/{aid}/approve", headers={"X-Token":
```

```
ed}).status_code == 403
    r = client.post(f"/api/admin/advisories/{aid}/approve", headers={"X-Token":
ap}).json()
    assert r["recipients"] == 2 and r["sms_fallback"] == 1
    # the published advisory is now retrievable by the assistant
    ans = client.post("/api/chat", json={"text": "Is there debris near
Sonprayag?"}).json()
    assert any("Road closed" in s["title"] for s in ans["sources"])
```

CHAPTER 6
SCREENSHOTS

All screenshots below were captured from the running prototype (python run.sh, demo data loaded) using an automated browser. The demo state includes an Orange rainfall alert in Rudraprayag district (85 mm), a restricted segment near Joshimath and five subscribed trips.

6.1 Traveller Web App

Assistant. A registration question is answered from the official content with its source and date. A Hindi crowd question is answered in Hindi with the live 7-day forecast. A route question returns the live route risk (Guptkashi to Sonprayag is Red because of the Orange alert, heavy rain and high susceptibility) with the safe halt Guptkashi. The right panel shows the detected intent, confidence, answer mode and sources.

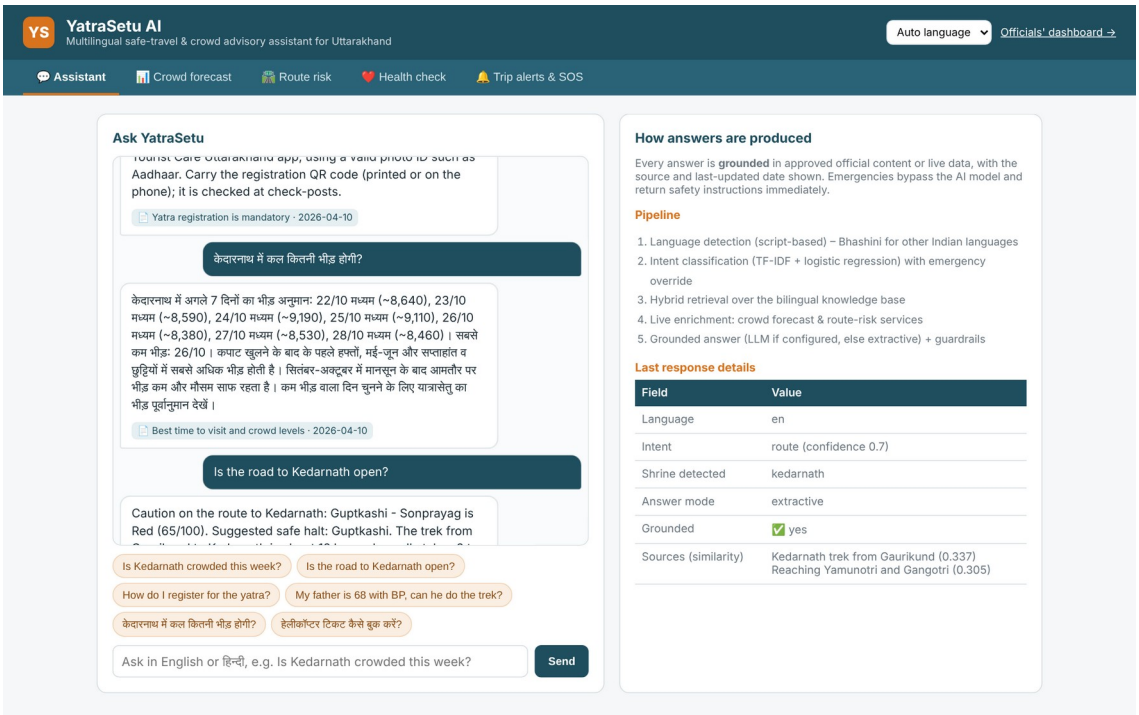


Figure 6.1: Assistant answering registration, Hindi crowd and route questions

An emergency message is answered immediately with 112/108 instructions and a call button, without using the AI model. An unrelated question (GDP of Brazil) is rejected with a safe fallback instead of a loosely related answer.

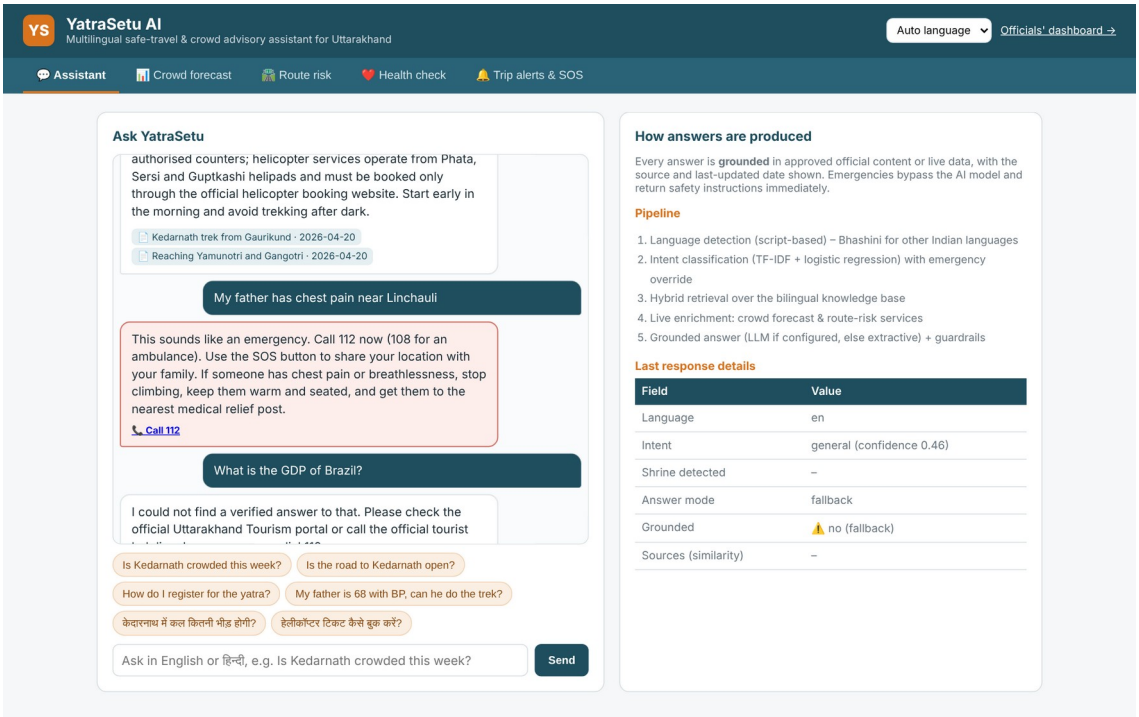


Figure 6.2: Emergency override and off-topic fallback

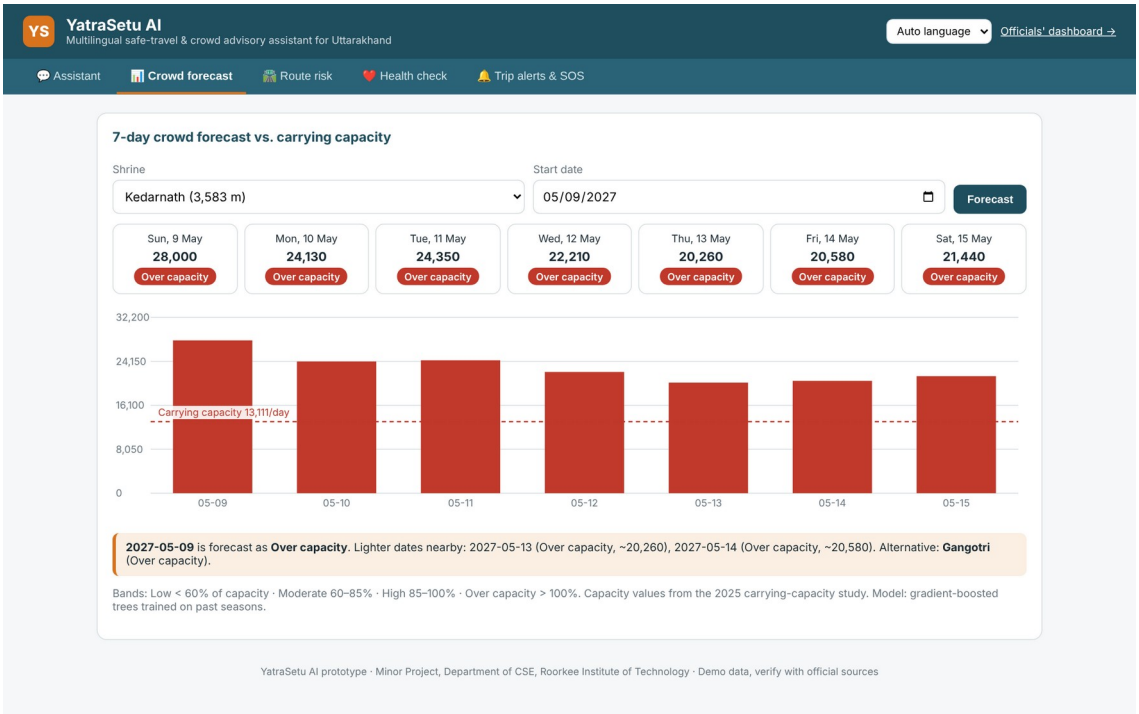


Figure 6.3: Crowd forecast for Kedarnath for the opening week of 2027 against carrying capacity

The crowd forecast for the expected opening week of the 2027 season shows every day over the carrying capacity of 13,111, with the opening day highest. The system suggests the lightest nearby dates and an alternative shrine.

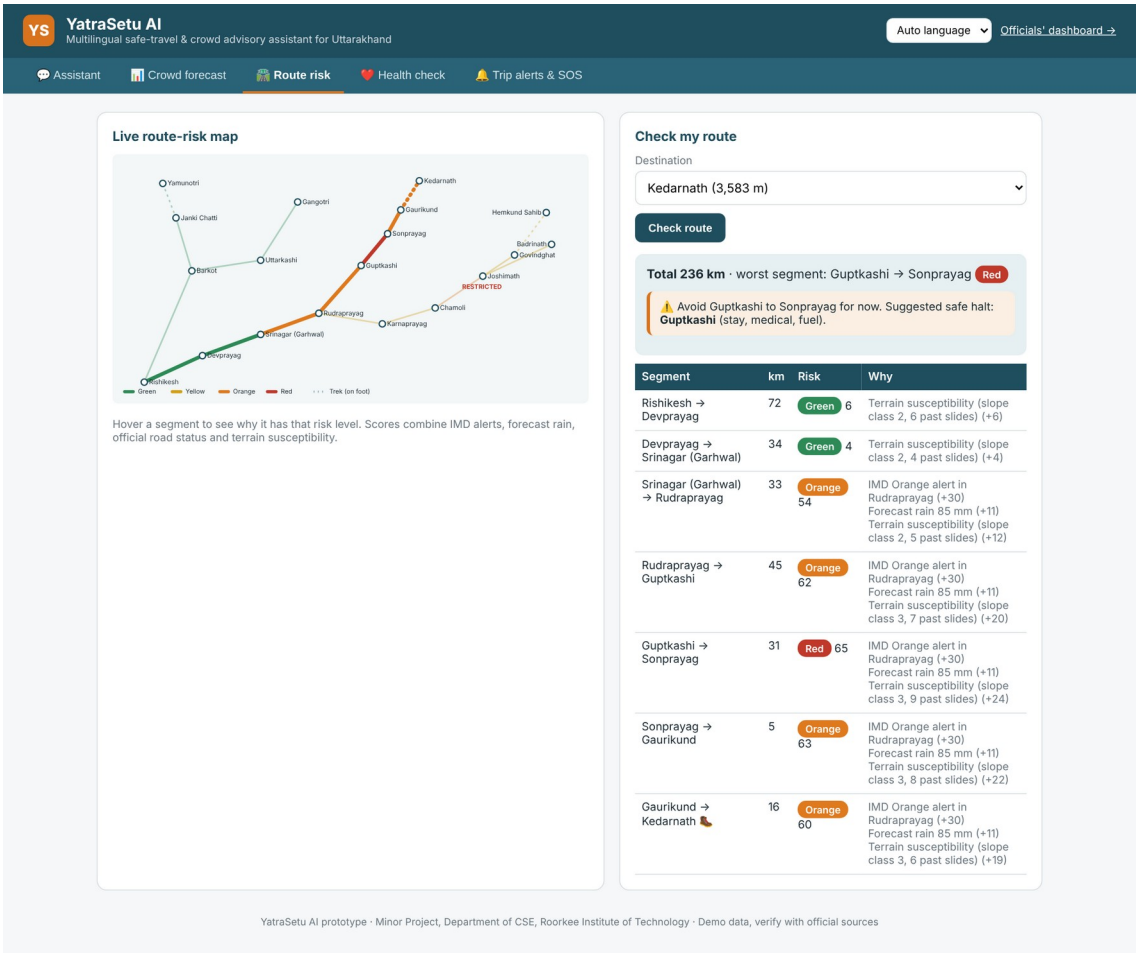


Figure 6.4: Live route-risk map and explained segment scores for the Kedarnath route

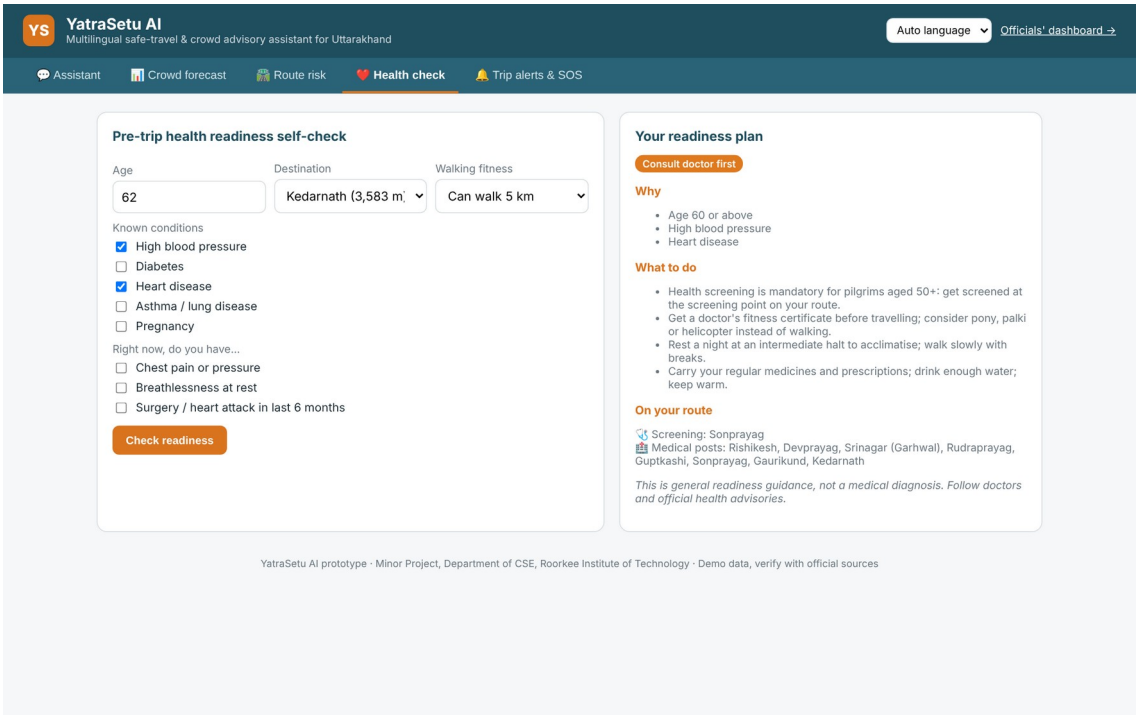


Figure 6.5: Health-readiness self-check for a 62-year-old pilgrim with high BP and heart disease

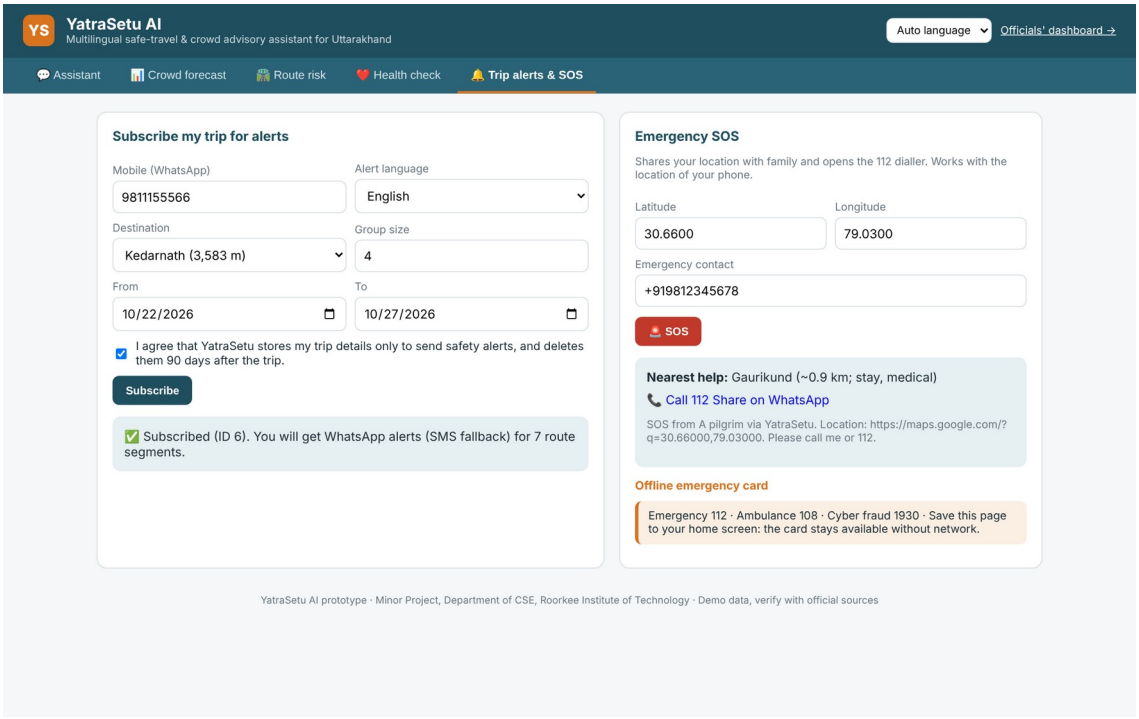


Figure 6.6: Trip subscription with consent, and SOS with nearest help

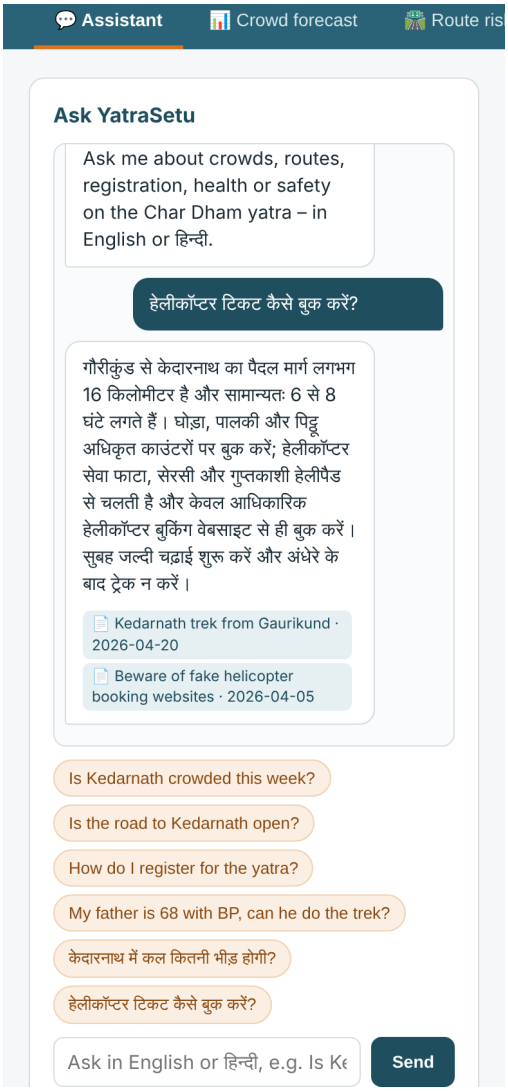


Figure 6.7: Mobile view: Hindi question about helicopter booking

6.2 Officials' Dashboard

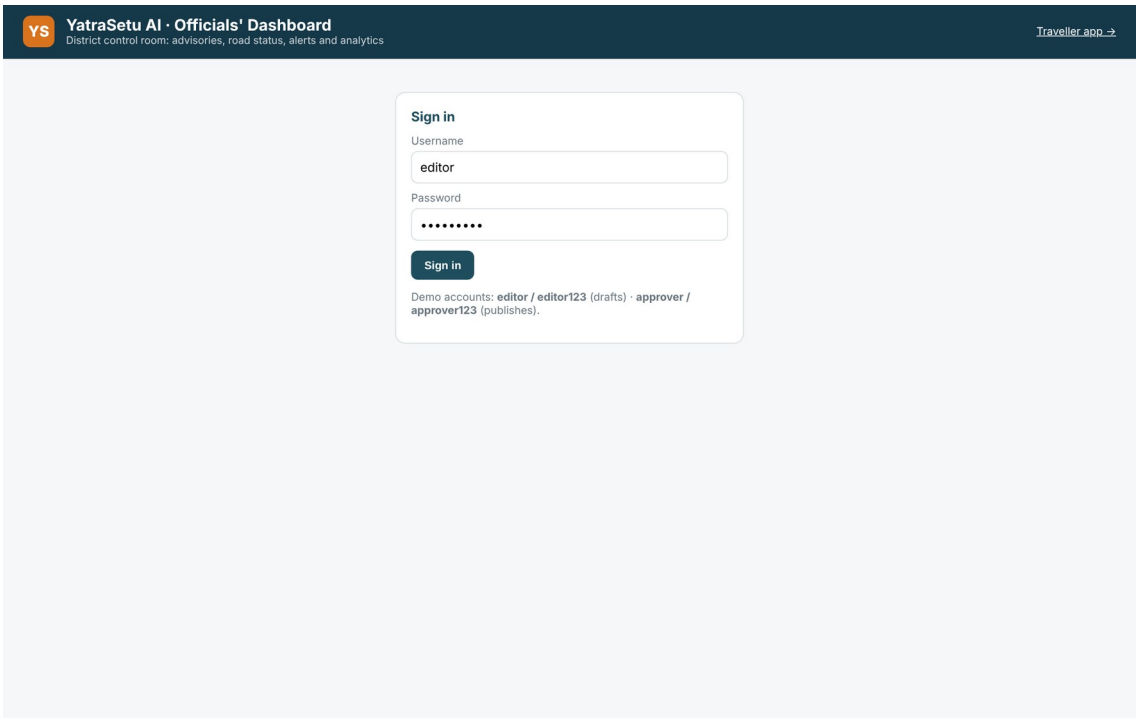


Figure 6.8: Dashboard login (editor and approver roles)

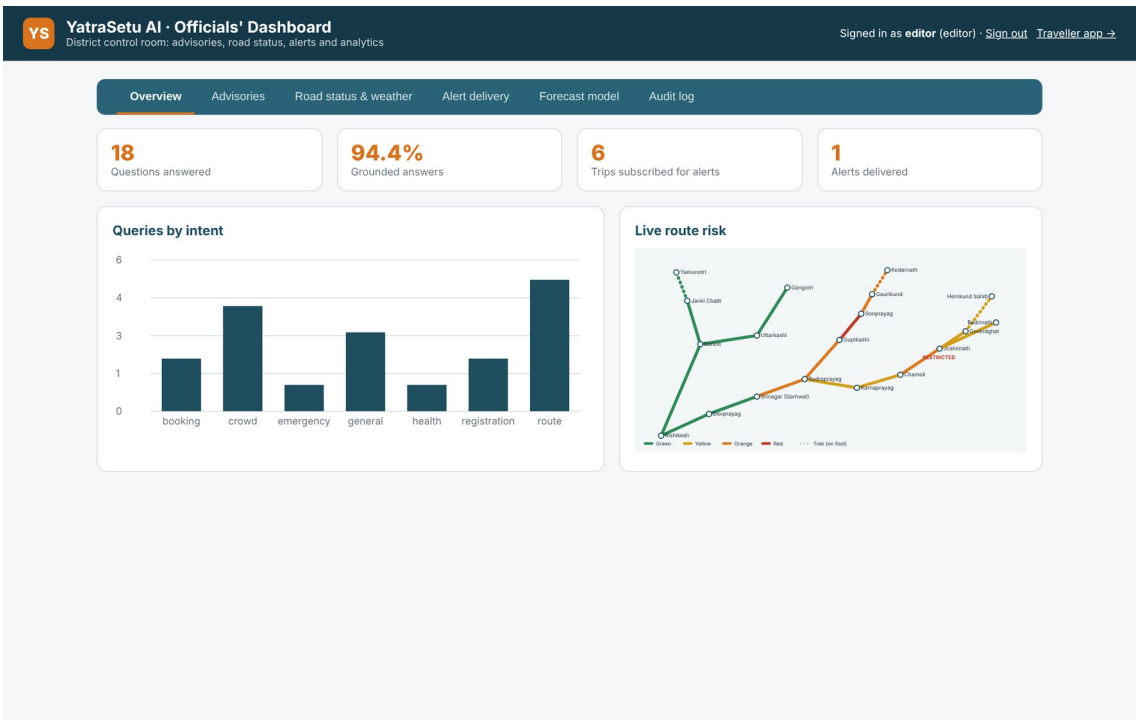


Figure 6.9: Dashboard overview: KPIs, questions by intent and live route risk

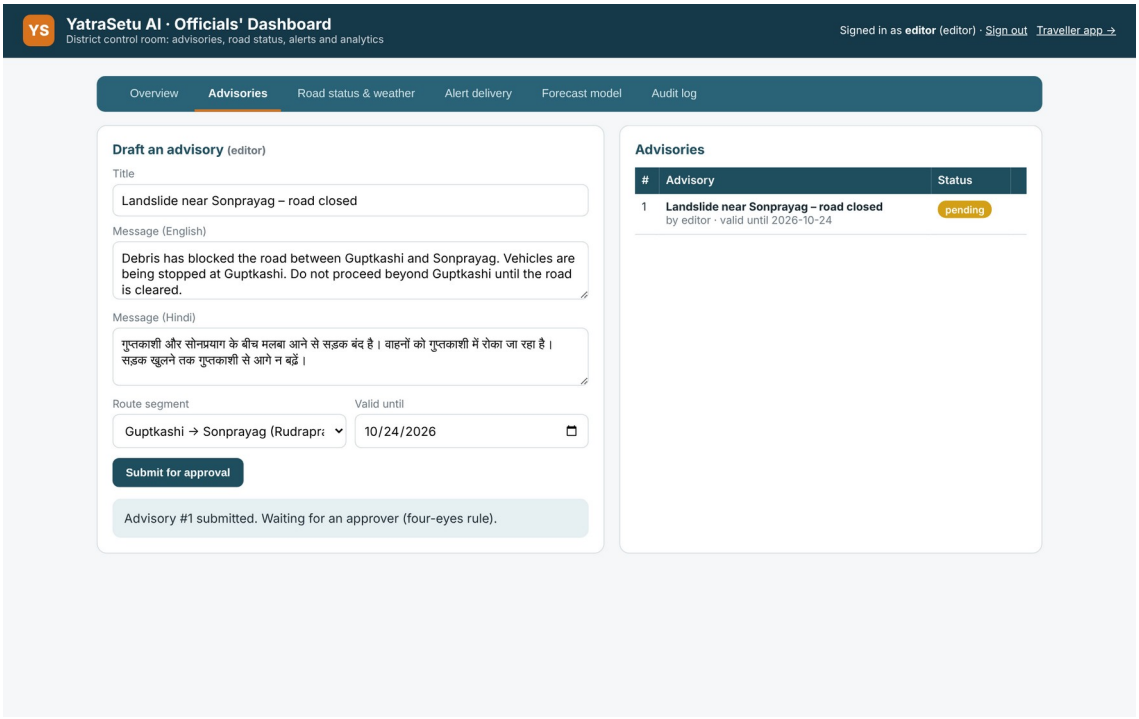


Figure 6.10: Editor drafts a bilingual advisory, which waits for approval

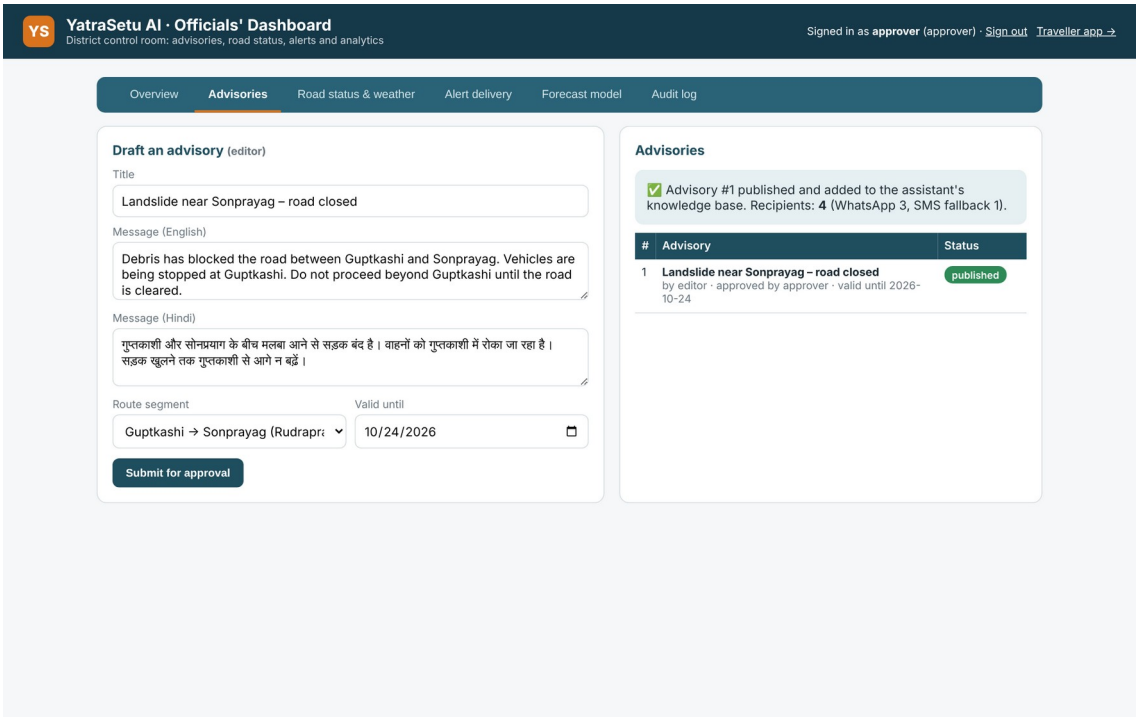


Figure 6.11: Approver publishes the advisory: 4 recipients, 3 by WhatsApp and 1 by SMS fallback

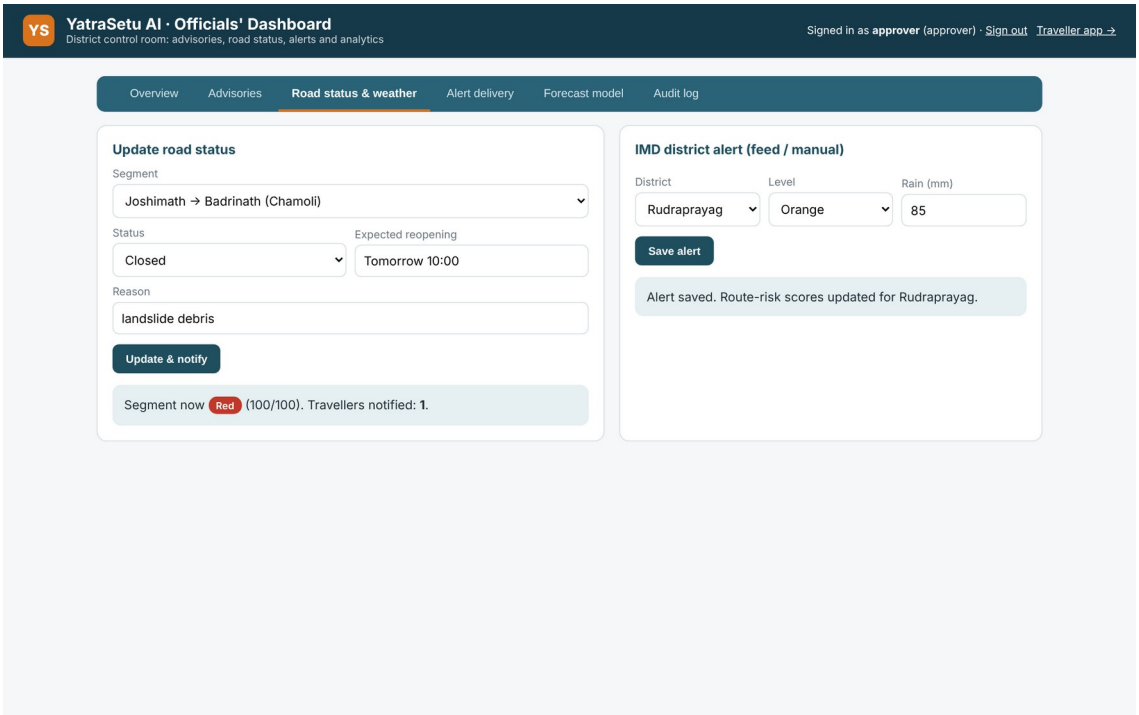


Figure 6.12: Road-status update and IMD district alert entry

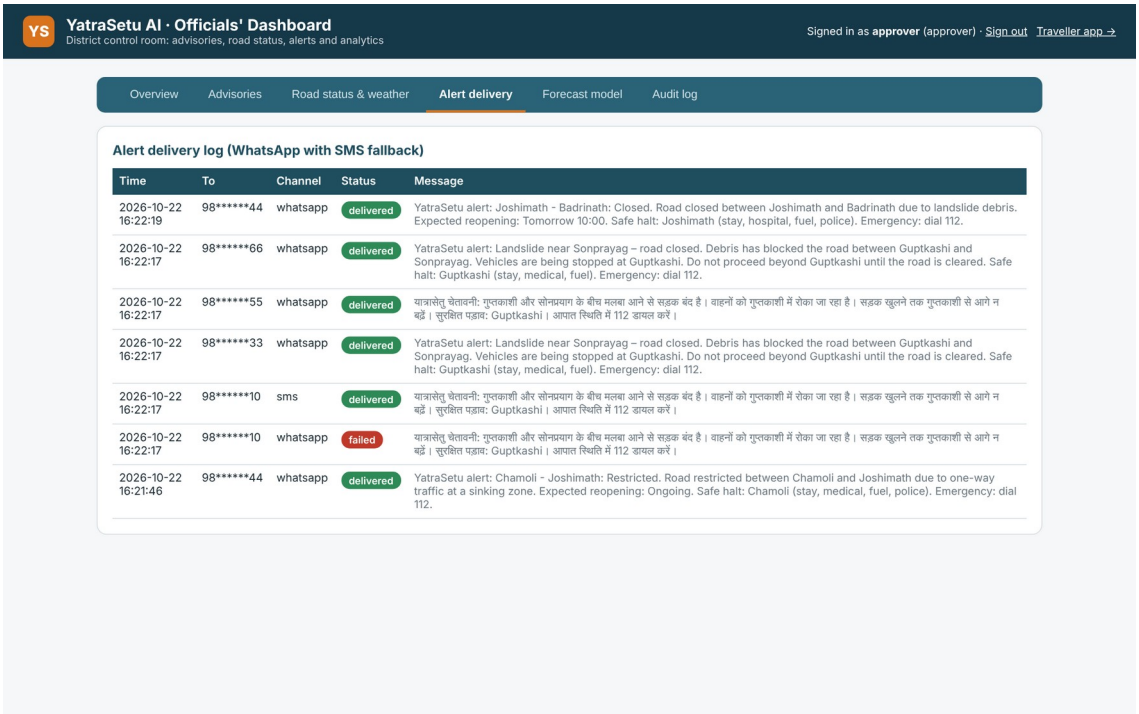


Figure 6.13: Alert delivery log in the traveller's language, with SMS fallback

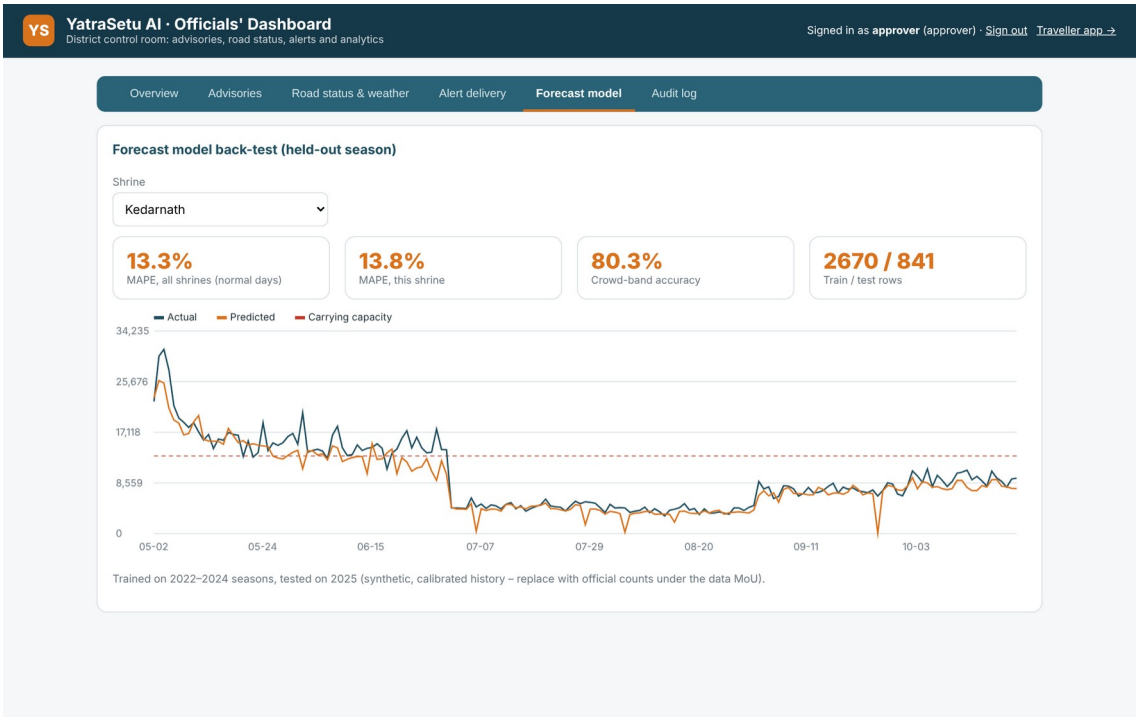


Figure 6.14: Forecast model back-test on the held-out 2025 season

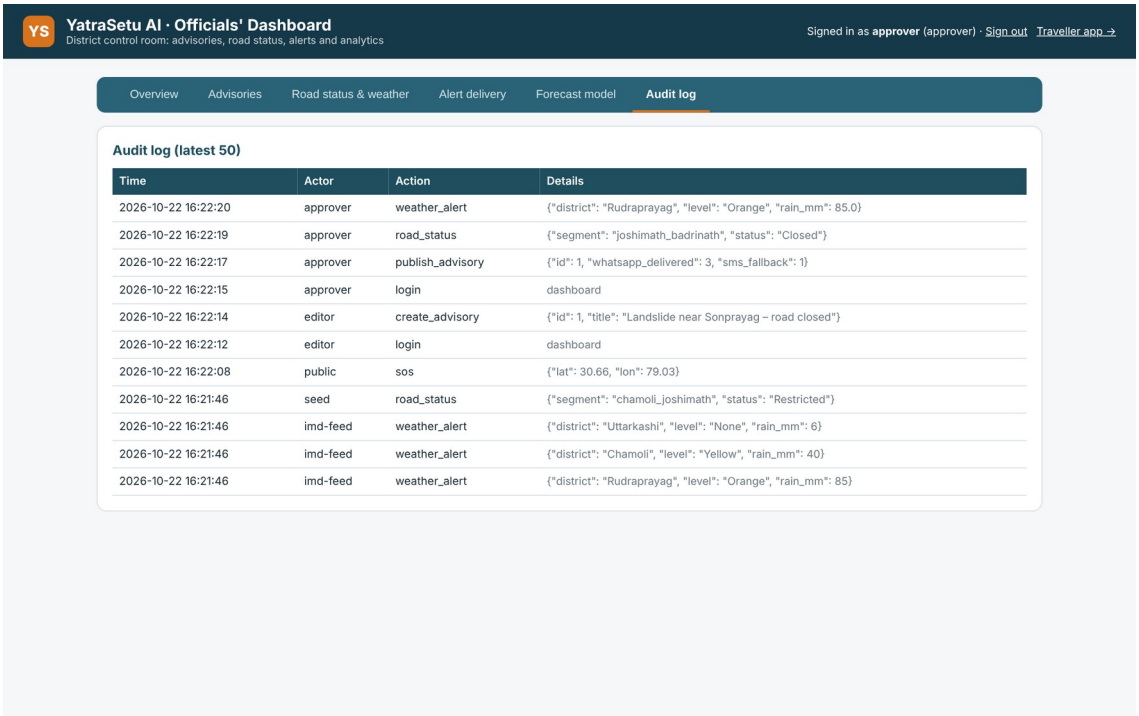


Figure 6.15: Audit log of official actions

6.3 Testing and Evaluation Results

6.3.1 Test cases

Table 6.1: Representative test cases

TC	Test	Expected result	Result
TC-01	Detect language of English, Hindi and Gujarati text	en, hi, gu	Pass
TC-02	"my father has chest pain" / "हम फंस गए हैं"	Intent = emergency	Pass
TC-03	Extract shrine from "is kedar crowded" and age from "68 years"	kedarnath, 68	Pass
TC-04	Forecast 7 days in season; MAPE on held-out season	7 positive values; MAPE <= 20%	Pass (13.3%)
TC-05	Forecast in January (season closed)	All days Closed	Pass
TC-06	Closed segment	Score 100, level Red	Pass
TC-07	Risk with Red alert and 140 mm rain vs. calm	Higher score, Orange/Red	Pass
TC-08	Safe halt for Sonprayag to Gaurikund	Town with stay + medical	Pass
TC-09	Health check with chest pain	Do not trek now	Pass
TC-10	68 years, BP + heart disease	Consult doctor first; Sonprayag screening	Pass
TC-11	LLM answer with number not in passages	Rejected by guardrail	Pass
TC-12	Ask registration question via API	Grounded answer with registration source	Pass
TC-13	Ask in Hindi	Answer in Devanagari	Pass
TC-14	Off-topic question	Fallback with 112	Pass
TC-15	Subscribe without consent	HTTP 400	Pass
TC-16	Approver drafts / editor approves / author self-approval	HTTP 403 / 403; approval by approver succeeds	Pass
TC-17	Broadcast to 2 subscribers, one failing on WhatsApp	2 recipients, 1 SMS fallback; advisory answerable	Pass
TC-18	Dashboard API without login	HTTP 401	Pass

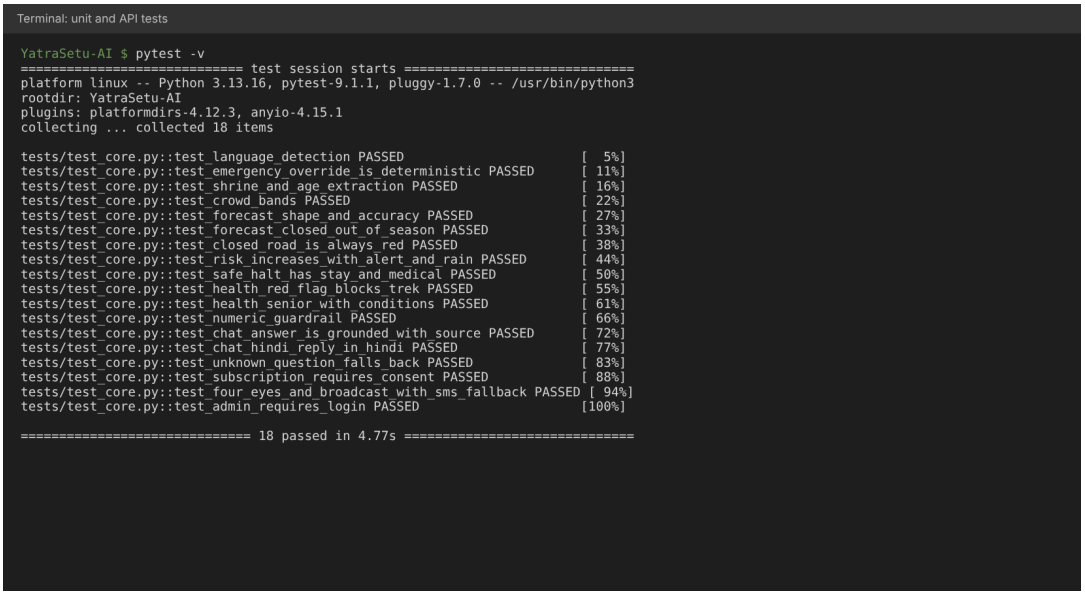


Figure 6.16: Automated test run: 18 tests passed

6.3.2 AI component evaluation

The evaluation script measures the AI components on data not used for training: a gold set of 40 bilingual questions (27 English, 13 Hindi) written independently of the intent-training examples, 10 emergency phrasings, 10 off-topic questions and the held-out 2025 forecasting season.

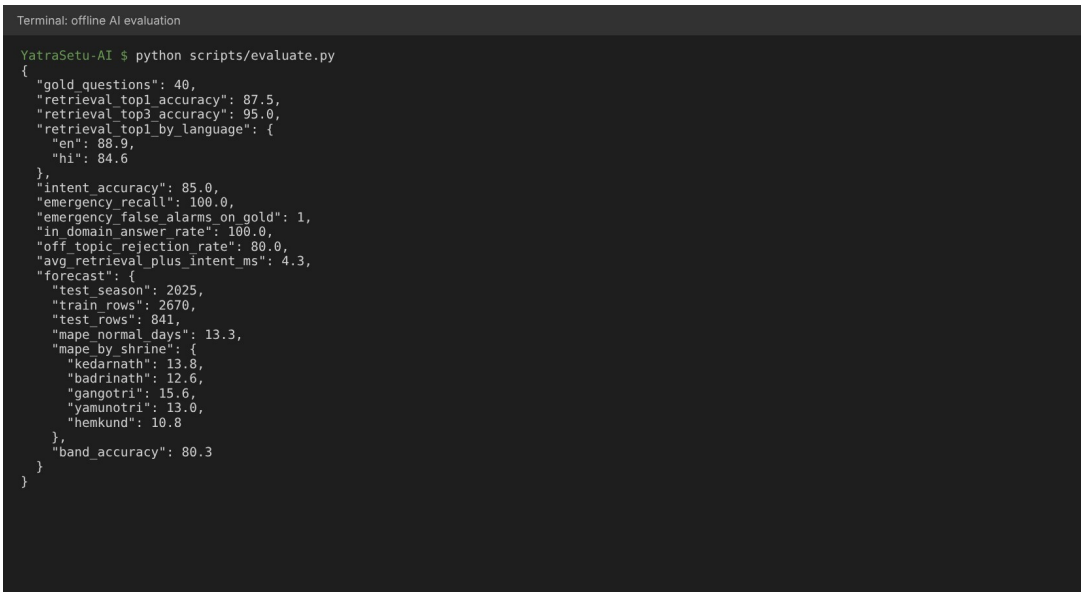


Figure 6.17: Output of the offline evaluation script

Table 6.2: Evaluation results

Component	Metric	Result	Target
Retrieval	Top-1 / top-3 accuracy (40 questions)	87.5% / 95.0%	Top-3 >= 90%
Retrieval	Top-1 accuracy, English / Hindi	88.9% / 84.6%	-

Component	Metric	Result	Target
Intent classifier	Accuracy on unseen questions	85.0%	-
Emergency override	Recall on 10 emergency messages	100%	100%
Emergency override	False alarms on 40 normal questions	1 ("emergency number" question)	Low
Domain gate	In-domain answered / off-topic rejected	100% / 80%	-
Forecasting	MAPE, normal days, all shrines (2025)	13.3%	<= 20%
Forecasting	MAPE by shrine (Kedarnath, Badrinath, Gangotri, Yamunotri, Hemkund)	13.8, 12.6, 15.6, 13.0, 10.8%	<= 20%
Forecasting	Crowd-band accuracy	80.3%	-
Performance	Retrieval + intent time per question	about 4.3 ms	< 4 s total

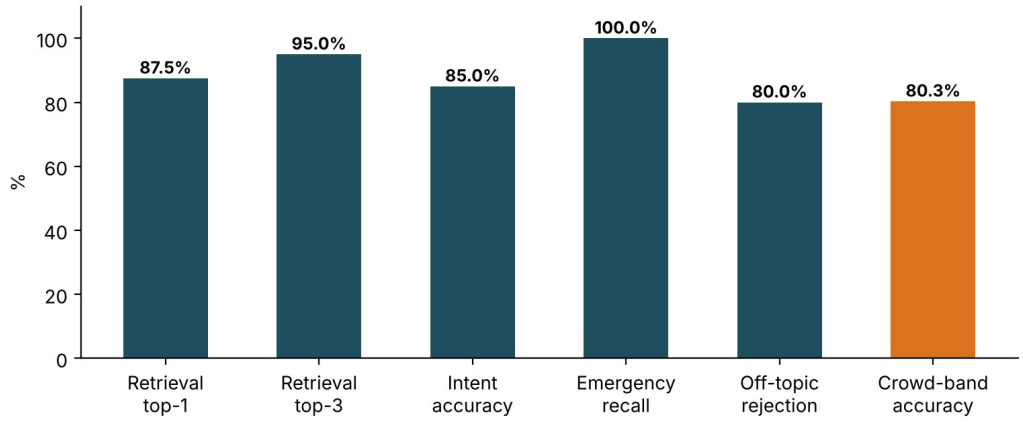


Figure 6.18: Evaluation results of the AI components

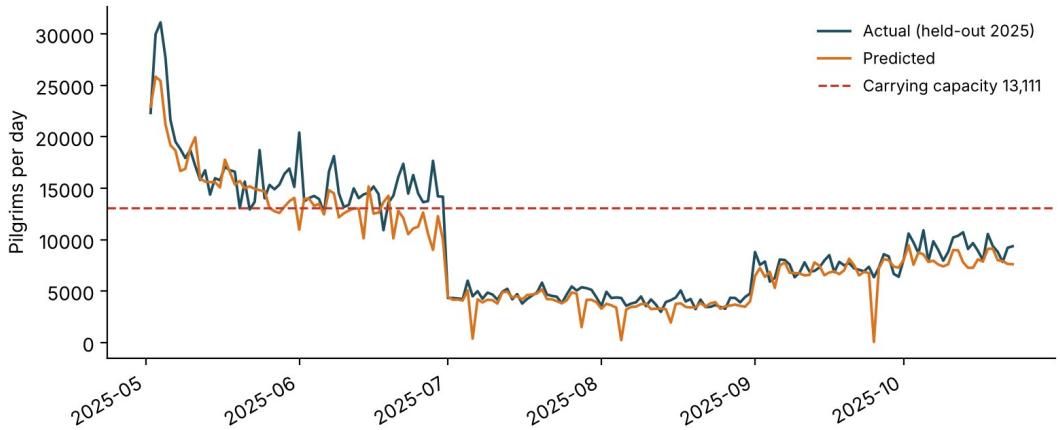


Figure 6.19: Kedarnath forecast vs. actual on the held-out 2025 season (synthetic history)

CHAPTER 7
CONCLUSION

7.1 Conclusion

This project set out to address a real and recurring problem in Uttarakhand: millions of pilgrims make safety-critical travel decisions with fragmented, generic information, while officials lack tools to forecast crowds and warn the right people in time. **YatraSetu AI** shows that a small, well-designed system built from open components can close much of this gap.

The working prototype combines a bilingual retrieval-grounded assistant with a deterministic emergency path, a gradient-boosted crowd forecaster linked to carrying capacity, an explainable route-risk model with safe-halt advice, a health-readiness module, consent-based trip alerts with SMS fallback, and an officials' dashboard with four-eyes approval and a full audit trail. All 18 automated tests pass, retrieval reaches 95% top-3 accuracy on a bilingual gold set, every emergency phrasing was caught, and the forecaster achieves 13.3% MAPE on a held-out season. The system runs on an ordinary laptop with no external services, and can be connected to Bhashini, an LLM and the WhatsApp Business Platform through configuration.

7.2 Achievement of Objectives

Table 7.1: Objectives versus achievement

Objective	Target	Achieved
O1 Bilingual grounded assistant	Top-3 retrieval >= 90%; sources shown	95.0% top-3; source and date on every grounded answer
O2 Crowd forecast vs. capacity	MAPE <= 20%	13.3% MAPE; 80.3% band accuracy; alternatives suggested
O3 Route risk and alerts	Automatic alerts for Orange/Red; SMS fallback	Implemented and tested (TC-06 to TC-08, TC-17)
O4 Health check and SOS	Correct categories; nearest help	Implemented and tested (TC-09, TC-10)
O5 Dashboard with four-eyes approval	Self-approval blocked; advisory immediately answerable	Implemented and tested (TC-16, TC-17)
O6 Low-cost, private, replicable	Offline; consent; hashed phone numbers	Runs offline; open-source; one-command setup

7.3 Limitations

- The footfall history is **synthetic** (calibrated to published totals); forecast accuracy must be re-validated on official daily counts.
- The knowledge base is a demonstration set of 13 documents; a production system needs content curated and maintained with the tourism, health and police departments.
- Hindi and English are supported end to end; other languages depend on Bhashini credentials, and voice input/output is not yet implemented.
- WhatsApp and SMS delivery are simulated through an outbox; live integration requires business verification and DLT-registered templates.
- Route attributes (slope class, landslide history) are illustrative planning values, not survey data.
- The extractive answer mode can include less relevant sentences for broad questions; the optional LLM mode improves fluency but needs an API and careful evaluation.

7.4 Future Scope

- Sign a data-sharing agreement to train on official daily footfall and registration data, and add real IMD feed integration.
- Add voice notes (Bhashini ASR/TTS) and an IVR line for feature-phone users; research Garhwali and Kumaoni language support.
- Use multilingual dense embeddings and re-ranking to improve retrieval, and evaluate an LLM-based answer mode with human reviewers.
- Integrate satellite rainfall nowcasts and landslide susceptibility maps (as in LHASA) for finer route-risk estimates.
- Pilot with district control rooms of Rudraprayag and Chamoli during a yatra season and measure impact on crowd peaks and response times.
- Extend to Kanwar Yatra, Adi Kailash, Valley of Flowers and to other Himalayan states.

CHAPTER 8
BIBLIOGRAPHY

References are arranged in alphabetical order of the first author's surname (or organisation).

[1] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P. and Amodei, D. et al., Language Models are Few-Shot Learners, Advances in Neural Information Processing Systems, 33, 1877-1901, 2020.

[2] Business Today, Rain fury in Uttarakhand: Cloudburst in Chamoli, 2,500 tourists stranded in Mussoorie, restoration work underway, 18 September 2025. Available: https://www.businesstoday.in

[3] Counterview, Rising footfall, rising risks: SDC Foundation seeks focus on safety and carrying capacity in Char Dham Yatra, December 2025. Available: https://www.counterview.in/2025/12/rising-footfall-rising-risks-sdc-seeks.html

[4] Deccan Herald, CM Dhami launches Uttarakhand Bhookamp Alert app, August 2021. Available: https://www.deccanherald.com

[5] Down To Earth, Tourism in Uttarakhand needs regulation. Available: https://www.downtoearth.org.in/environment/tourism-in-uttarakhand-needs-regulation-41499

[6] ETV Bharat, Why Char Dham witnesses high death toll? 56 devotees died in 28 days, 28 May 2025. Available: https://www.etvbharat.com

[7] ETV Bharat, Uttarakhand records all-time high tourist footfall in 2025, crosses 6 crore mark, February 2026 (cited as ETV Bharat, 2026a). Available: https://www.etvbharat.com

[8] ETV Bharat, Adequate attention needs to be paid to carrying capacity of Chardham Yatra shrines, May 2026 (cited as ETV Bharat, 2026b). Available: https://www.etvbharat.com

[9] Friedman, J.H., Greedy Function Approximation: A Gradient Boosting Machine, The Annals of Statistics, 29(5), 1189-1232, 2001.

[10] Froude, M.J. and Petley, D.N., Global fatal landslide occurrence from 2004 to 2016, Natural Hazards and Earth System Sciences, 18, 2161-2181, 2018.

[11] Gala, J., Chitale, P.A., Raghavan, A.K., Gumma, V., Doddapaneni, S., Kumar, A., Khapra, M.M., Dabre, R. and Kunchukuttan, A. et al., IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for all 22 Scheduled Indian Languages, Transactions on Machine Learning Research, 2023.

[12] Government of India, Digital Personal Data Protection Act, 2023 (No. 22 of 2023), Ministry of Electronics and Information Technology, 2023.

[13] Government of Uttarakhand, Registration and Tourist Care System: Registration Process Steps, Uttarakhand Tourism Development Board, 2024. Available: https://registrationandtouristcare.uk.gov.in

[14] Gretzel, U., Sigala, M., Xiang, Z. and Koo, C., Smart tourism: foundations and developments, Electronic Markets, 25(3), 179-188, 2015.

[15] Hyndman, R.J. and Athanasopoulos, G., Forecasting: Principles and Practice, 3rd edition, OTexts, Melbourne, 2021.

[16] IEEE, IEEE Recommended Practice for Software Requirements Specifications, IEEE Std 830-1998, 1998.

[17] ISO/IEC/IEEE, Systems and software engineering: Life cycle processes, Requirements engineering, ISO/IEC/IEEE 29148:2018, 2018.

[18] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y.J., Madotto, A. and Fung, P., Survey of Hallucination in Natural Language Generation, ACM Computing Surveys, 55(12), 1-38, 2023.

[19] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T.-Y., LightGBM: A Highly Efficient Gradient Boosting Decision Tree, Advances in Neural Information Processing Systems, 30, 3146-3154, 2017.

[20] Kirschbaum, D. and Stanley, T., Satellite-Based Assessment of Rainfall-Triggered Landslide Hazard for Situational Awareness, Earth's Future, 6(3), 505-523, 2018.

[21] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S. and Kiela, D., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, Advances in Neural Information Processing Systems, 33, 9459-9474, 2020.

[22] Manning, C.D., Raghavan, P. and Schutze, H., Introduction to Information Retrieval, Cambridge University Press, 2008.

[23] News On AIR, Heavy rain triggers landslides in Uttarakhand; Char Dham Yatra halted, 3 July 2025. Available: https://www.newsonair.gov.in

[24] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O. and Duchesnay, E. et al., Scikit-learn: Machine Learning in Python, Journal of Machine Learning Research, 12, 2825-2830, 2011.

[25] Pressman, R.S. and Maxim, B.R., Software Engineering: A Practitioner's Approach, 9th edition, McGraw-Hill Education, 2020.

[26] Rautela, P., Lessons learnt from the Deluge of Kedarnath, Uttarakhand, India, Asian Journal of Environment and Disaster Management, 5(2), 2013.

[27] Robertson, S. and Zaragoza, H., The Probabilistic Relevance Framework: BM25 and Beyond, Foundations and Trends in Information Retrieval, 3(4), 333-389, 2009.

[28] Salton, G. and Buckley, C., Term-weighting approaches in automatic text retrieval, Information Processing and Management, 24(5), 513-523, 1988.

[29] Sommerville, I., Software Engineering, 10th edition, Pearson, 2016.

[30] Taylor, S.J. and Letham, B., Forecasting at Scale, The American Statistician, 72(1), 37-45, 2018.

[31] The Quint, Char Dham Yatra: Uttarakhand Govt Fixes Daily Cap on Pilgrims Visiting Shrines, May 2022. Available: https://www.thequint.com

[32] The Week, Char Dham Yatra breaks records with unprecedented number of pilgrims, 24 June 2025. Available: https://www.theweek.in

[33] UNICEF, Bhashini AI: Making Languages More Accessible with Digital Technology, UNICEF Digital Impact. Available: https://www.unicef.org/digitalimpact

[34] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I., Attention Is All You Need, Advances in Neural Information Processing Systems, 30, 5998-6008, 2017.

[35] W3C, Web Content Accessibility Guidelines (WCAG) 2.1, World Wide Web Consortium Recommendation, 2018.